



网络安全技术及应用

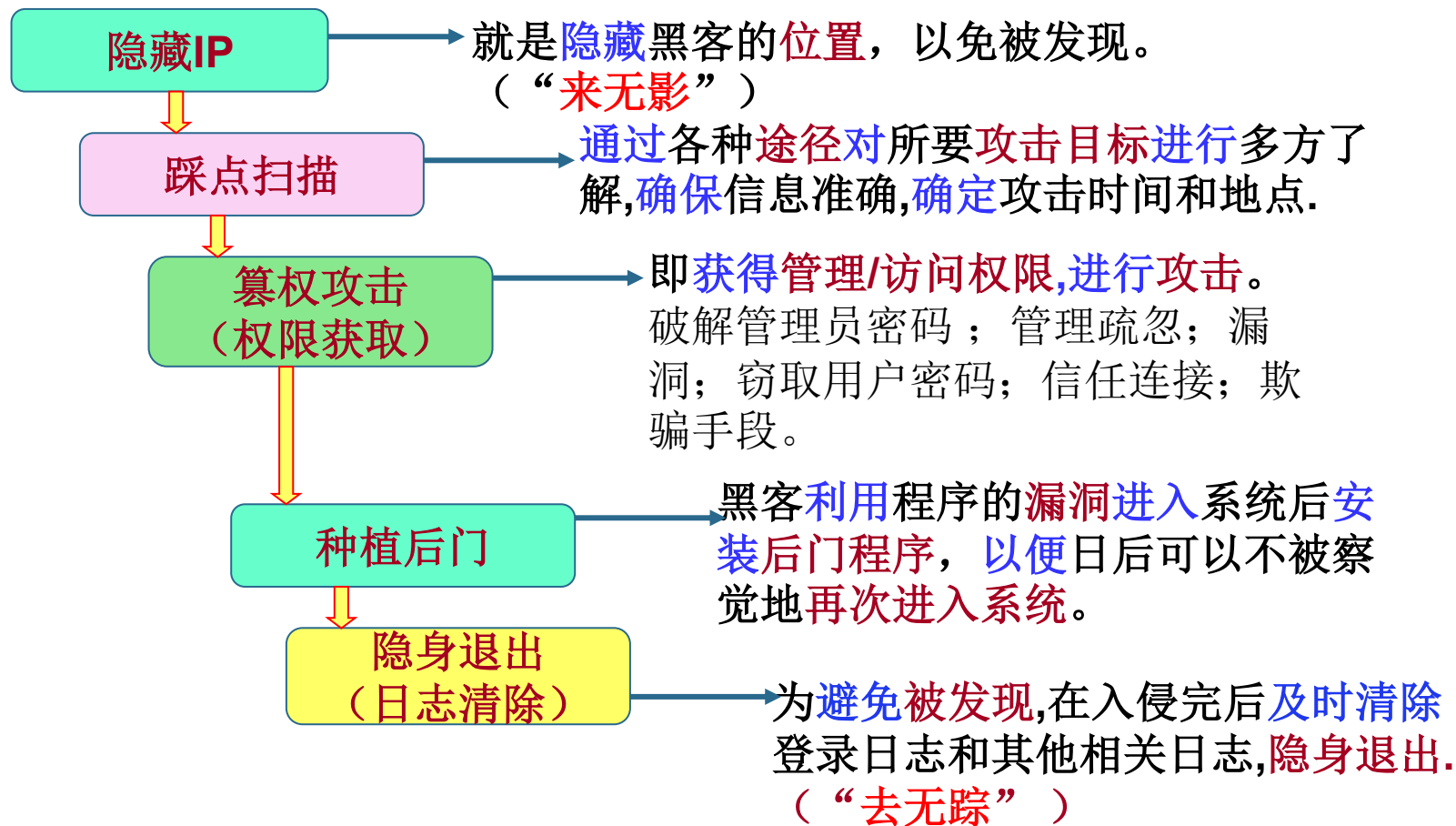
(第5版, 慕课微课版, 新形态)

贾铁军 何道敬 罗宜元 主编
古乐声 王威 张书台 副主编
王坚 陈国秦 参编

 机械工业出版社
CHINA MACHINE PRESS

第5章 密码与加密技术

回顾



回顾

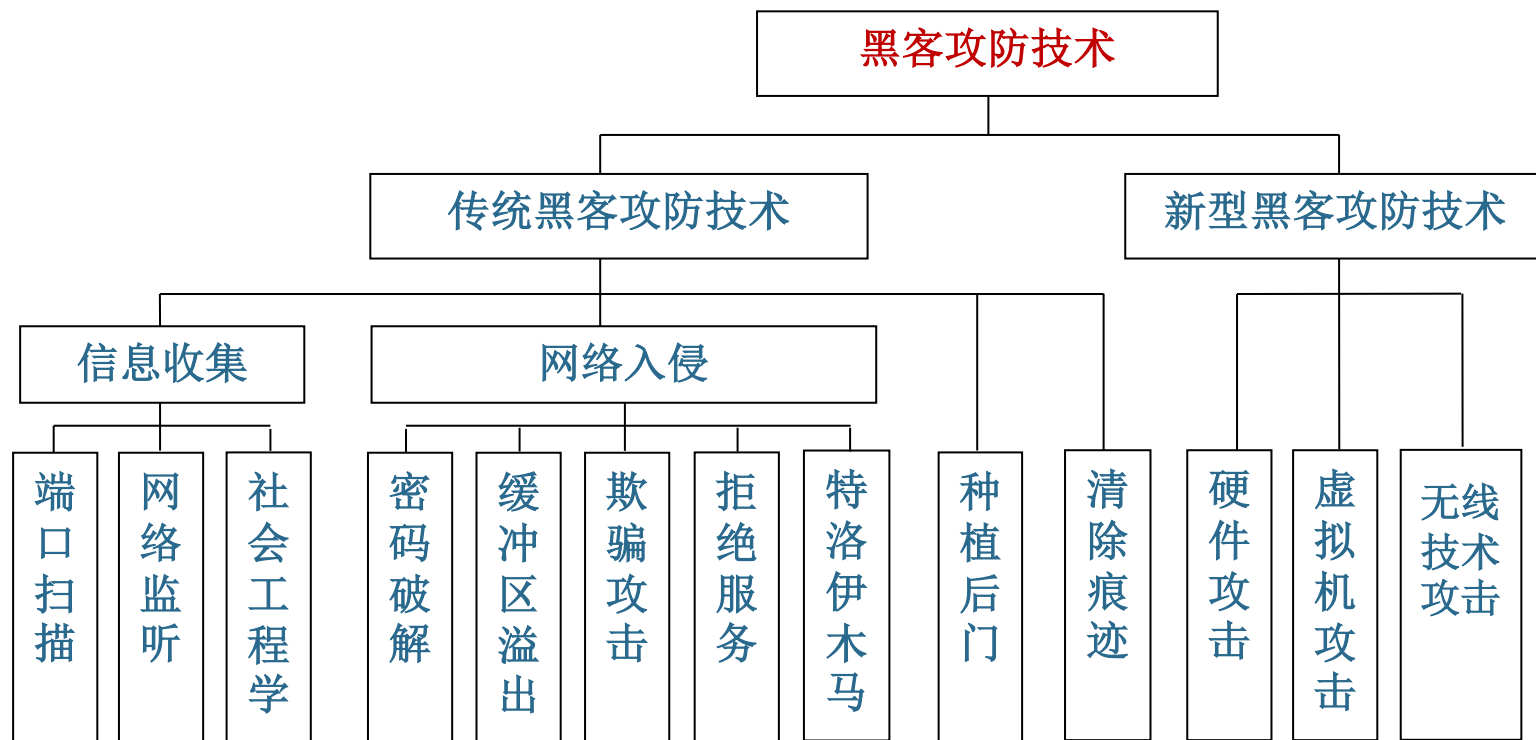


图4-2常用的黑客攻防技术

回顾

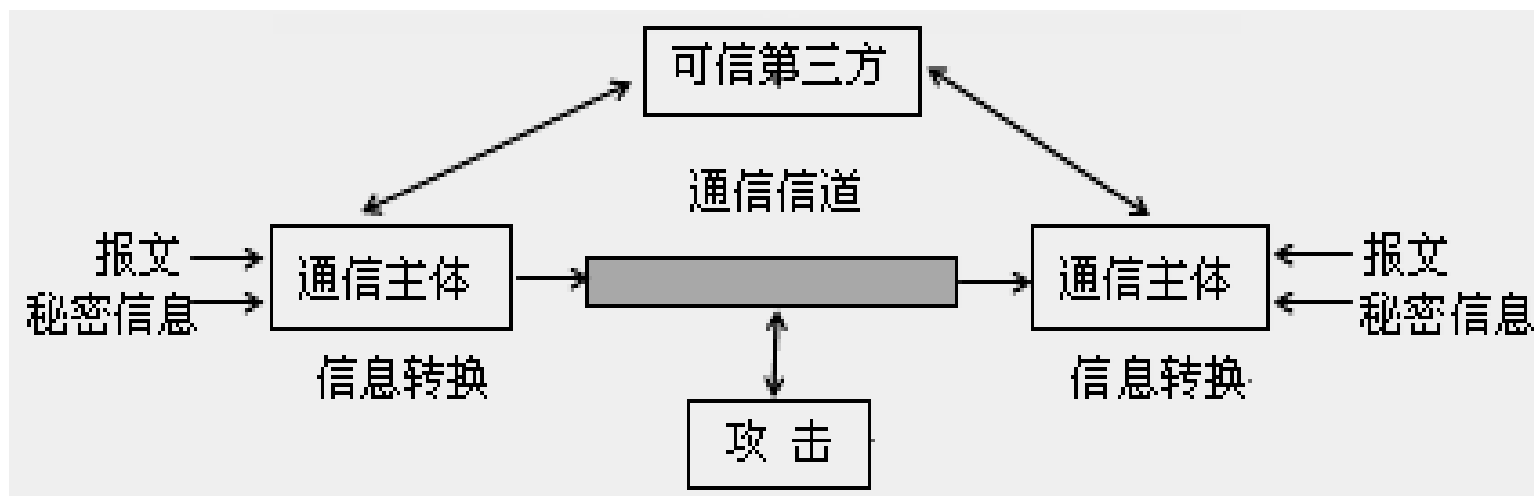


图1-7 网络安全常用模型

第5章 密码与加密技术

第5章 目录

1

5.1 密码技术概述

2

5.2 密码破译与密钥管理

3

5.3 实用密码技术概述

4

5.4 AES算法加密解密实验

5

5.5 本章小结



第5章 密码与加密技术

□ 教学目标

- 了解密码学的产生过程和发展历程
- 掌握密码学主要相关概念和加密方式 **重点**
- 理解密码破译常用方法和密钥管理
- 掌握常见的主要密码技术及其应用 **重点**



5.1 密码技术概述

引导案例

密码技术已经成为现代人类日常生活的一部分。如果离开了密码技术，人们的生活将会陷入停顿，整个互联网及其设备将无法运行，银行交易陷入停滞，公共交通将陷入瘫痪，任何人都可以访问我们的私有信息，那些不怀好意的人则会伤害我们。

研究密码技术的学问称为密码学，密码学保护人们通信和信息的安全，见于人们的日常生活中。例如，密码学保护了用户在ATM机存取款的安全性，基于密码学的HTTPS协议保护了用户在网上支付、收发网络邮件的安全性。

5.1 密码技术概述

- 最早的密码可追溯到4000多年前，在古埃及的尼罗河畔，一个贵族墓碑上的铭文并不是普通象形文字，而是变形的象形文字。
- 公元前5世纪，古斯巴达人最早使用了一种叫做“天书”的器械。将长长的带状羊皮纸缠绕在一根木棍上，书写情报。写完后，将羊皮纸从木棍上解下来展开，得到的是一纸杂乱无章的“天书”。对方收到羊皮纸，找一根相同粗细的棍子缠绕上去，才能看到情报内容。



5.1 密码技术概述

- 1470年，密码盘——莱昂·巴蒂斯塔·阿尔伯蒂发明了第一个用于加密的机械装置。它由一大一小两个同心圆组成，通常用内盘的字母取代外盘的字母进行密码编写。
- 凯撒密码。



5.1 密码技术概述

案例5-1

凯撒密码。根据古罗马历史学家苏维托尼乌斯的记载，凯撒曾用此方法对重要军事信息进行加密：“如果需要保密，信中使用暗号，即改变字母顺序，使局外人无法组成一个单词。如果想要读懂和理解它们的意思，得用第4个字母置换第1个字母，即以D代A，以此类推。”

a	b	c	d	e	f	g	h	i	j	k	l	m
d	e	f	g	h	i	j	k	l	m	n	o	p
n	o	p	q	r	s	t	u	v	w	x	y	z
p	r	s	t	u	v	w	x	y	z	a	b	c



5.1 密码技术概述

- 古代中国隐写术——矾书。“矾书”就是用明矾水写的书信。当水干后,纸上毫无字迹,把它弄湿后,字迹重新显现。
- 隐写术从严格意义上讲不属于密码学,但它其实也是自古以来一种常用的秘密通信手段。随着科学技术的发展,基于种种物理、化学等原理的隐写术也非常发达,被广泛应用于军事、外交和谍报战线。即使到了计算机网络时代,隐写术同密码术一样,也有了基于数字化技术的发展——如把信文隐藏在图像、声音或视频文件中。

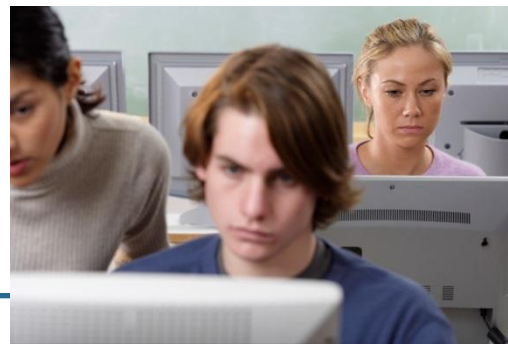
5.1.1 密码学发展历程

- 一战、二战中密码学被广泛应用并飞速发展。
- 1949年，**香农**（Shannon）**开创性**发表了**论文**《保密系统的通信原理》，为密码学建立了理论基础，从此成为一门科学。此后，对密码学的研究越来越多。
- 1976 年，密码学界发生两件**有影响力事情**。一是数据加密算法DES发布，二是Diffie和Hellman公开提出公钥密码学概念。DES 算法的发布是对称密码学的一个里程碑，而公钥密码学概念的出现也使密码学开辟了一个新方向。
- 此后，密码学从军事领域拓展成为公开学术研究方向。无论是对称密码学还是公钥密码学，都是为了解决数据的**保密性、完整性和认证性**这三个主要问题。



5.1.1 密码学发展历程

- **密码学**是研究信息保密性、完整性和认证性的科学，是数学和计算机的交叉学科，也是一门新兴并极有发展前景的学科。
- **密码学研究目的**保证数据的保密性、完整性和认证性。
- **数据的保密性**是指未经授权的用户不可获得保密数据。
- **数据的完整性**是验证数据在传输中未经授权篡改。
- **数据的认证(审查)性**是验证数据发送方的真实唯一性。



5.1.2 密码学相关概念

- 密码学包含两个互相对立的分支
- 研究编制密码的技术称为**密码编码学**（Cryptography），主要研究对数据进行变换的原理、手段和方法，**用于**密码体制设计。
- 研究破译密码的技术称为**密码分析学**（Cryptanalysis），主要研究内容如何破译密码算法。密码编制学和密码分析学共同组成**密码学**。



5.1.2 密码学相关概念

- 常用基本概念
- 在学习密码技术之前，首先定义一些术语。
- 明文是原始的信息（文件）（Plaintext，记为P）。
- 密文是明文经过变换加密后信息（Ciphertext，记为C）。
- 加密是从明文变成密文的过程（Enciphering，记为E）。
- 解密是密文还原成明文的过程（Deciphering，记为D）。



5.1.2 密码学相关概念

- **加密算法**（Encryption Algorithm）是实现加密所遵循的规则。**用于**对明文进行各种代换和变换生成密文。
- **解密算法**（Decryption Algorithm）是实现解密所遵循的规则，是加密算法的逆运行，由密文得到明文。
- **密钥**（Key，记为K）。为了有效地控制加密和解密算法的实现，密码体制中要有通信双方的专门的保密“信息”参与加密和解密操作，这种专门信息称为密钥。



5.1.2 密码学相关概念

- **加密协议**定义了如何使用加密、解密算法解决特定的任务。
- 发送消息的对象称作**发送方**（Sender）。
- 传送消息的预定接收对象称作**接收方**（Receiver）。
- 非授权进入计算机及其网络系统者称为**入侵者**（Intruder）。



5.1.2 密码学相关概念

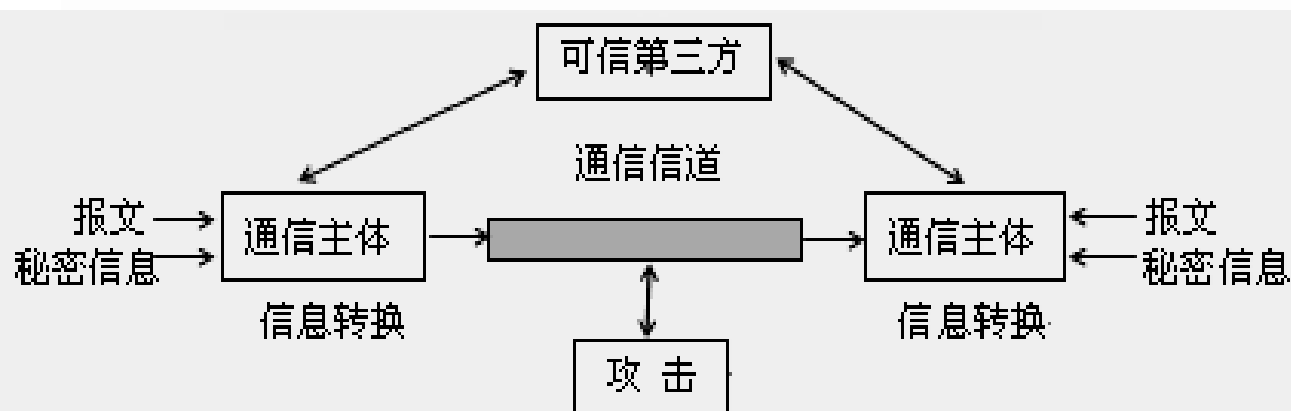
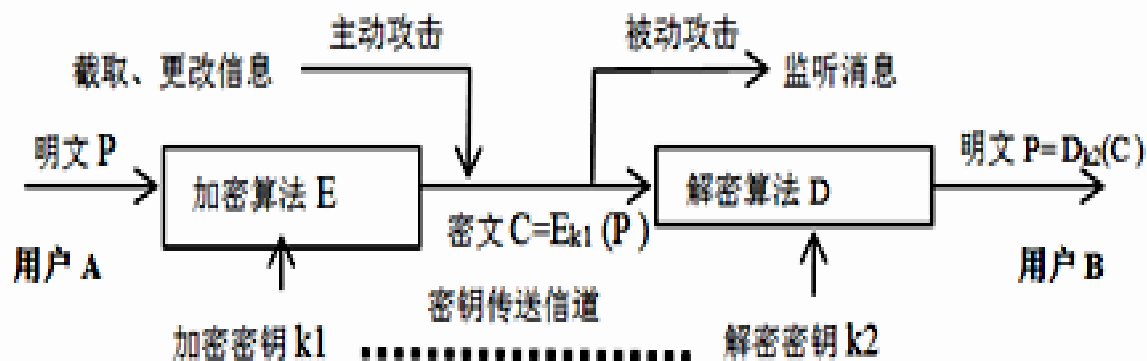
- 在消息传输和处理系统中，除了特定的接收者外，非授权者通过某种办法（如搭线窃听、电磁窃听、声音窃听等）窃取机密信息，称为**窃听者** (Eavesdropper)。
- **入侵者**主动侵扰系统，采用删除、更改、增添、重放、伪造等手段向系统注入假消息，这类攻击称作**主动攻击** (Active Attack)。
- 对一个密码体制采取截获密文进行分析，称作**被动攻击** (Passive Attack)。



5.1.2 密码学相关概念

■ 密码体制

- 任何一个密码体制至少包括5个组成部分：明文、密文、加密、解密算法及密钥。



5.1.2 密码学相关概念

- 一个**密码体制**的基本工作过程：**发送方**用加密密钥，通过加密算法，将明文信息加密成密文后发送出去；
- **接收方**在收到密文后，用解密密钥，通过解密算法将密文解密，恢复为明文。如果传输中有人窃取，只能得到无法理解的密文，从而对信息起到保密作用。



5.1.2 密码学相关概念

■ *Kerckhoff（柯克霍夫斯）原理

- 系统密文不可破译（在保护期内）；
- 系统的保密性不依赖于对加密算法的保密，而是依赖于密钥；
- 加密和解密算法适用于密钥空间中的所有密钥；
- 系统应该有良好的可用性，便于实现和使用。

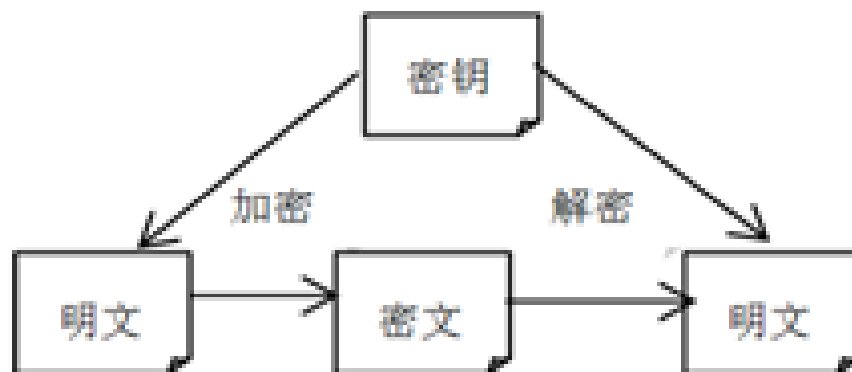


5.1.2 密码学相关概念

■ 密码体制的分类

■ 对称密码体制

- 加密、解密都需要密钥。如果加、解密密钥相同，这样的系统称为对称密钥密码体制(Symmetric Key System), 也称单钥密码体制。系统特点是加、解密的密钥是相同的、保密的。
- 常见的对称加密算法包括DES、3DES、AES等。

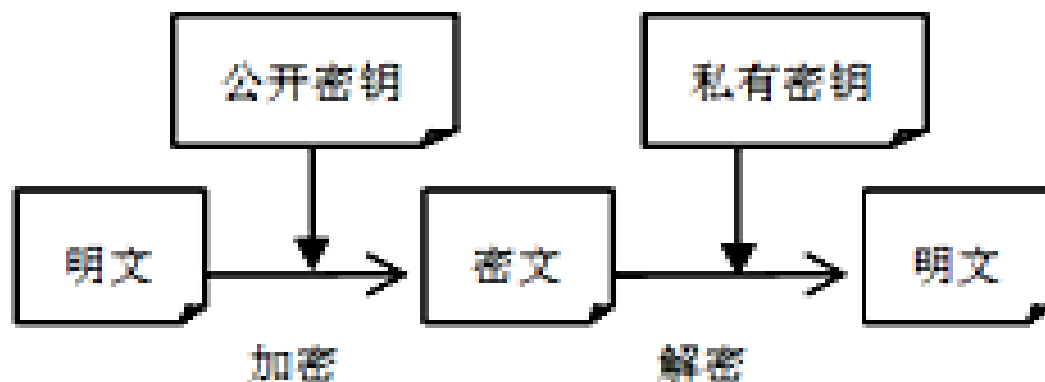


5.1.2 密码学相关概念

■ 非对称密码体制

■ 如果加、解密密钥不同，则这种系统是非对称密钥密码体制 (Non-Symmetric Key System)，又称双钥密码体制、公开密钥密码体制 (PKI)。特点：

- 1. 公开公钥，私钥由用户保存，保密管理相对简单；
- 2. 加密算法复杂，加密速度慢成本高(用于加密密钥)；
- 3. 非对称加密相比对称加密更安全可靠。
- 常见的非对称加密算法包括RSA、ECC等。



使用非对称密码体制时，应该使用谁生成的密钥？

5.1.2 密码学相关概念

■ 对称与非对称密码体制特性对比

表 5-1 对称密码体制与非对称密码体制特点对比

特征	对称密码体制	非对称密码体制
密钥的数目	单一密钥	密钥是成对的
密钥种类	密钥是秘密的	需要公开密钥和私有密钥
密钥管理	简单、不好管理	需要数字证书及可信任的第三方
计算速度	非常快	比较慢
用途	加密大块数据	加密少量数据或数字签名

5.1.2 密码学相关概念

■ 对称密码体制

■ 优点:

- **加解密速度快**: 对称加密算法的加解密过程相对简单, 因此处理速度较快, 适合加密大量数据;
- **加密强度高**: 对称加密算法的加密强度较高, 能够提供较好的安全性。

■ 缺点:

- **密钥管理复杂**: 对称加密算法中, 密钥的管理和分发是一个大问题。需要确保密钥在传输过程中不被窃取或泄露, 增加了管理复杂性和成本;
- **不适用于多方通信**: 在多方通信中, 每个通信方都需要一个独特的密钥, 导致密钥数量爆炸性增长, 管理难度大。

■ 非对称密码体制

■ 优点:

- **密钥管理方便**: 非对称加密算法使用公钥和私钥, 公钥可以公开, 而私钥由持有者保管。这种机制简化了密钥的分发和管理;
- **实现数字签名**: 非对称加密算法可以方便地实现数字签名功能, 确保数据的完整性和不可抵赖性;
- **高安全性**: 非对称加密算法的安全性更高, 能够抵御某些类型的攻击。

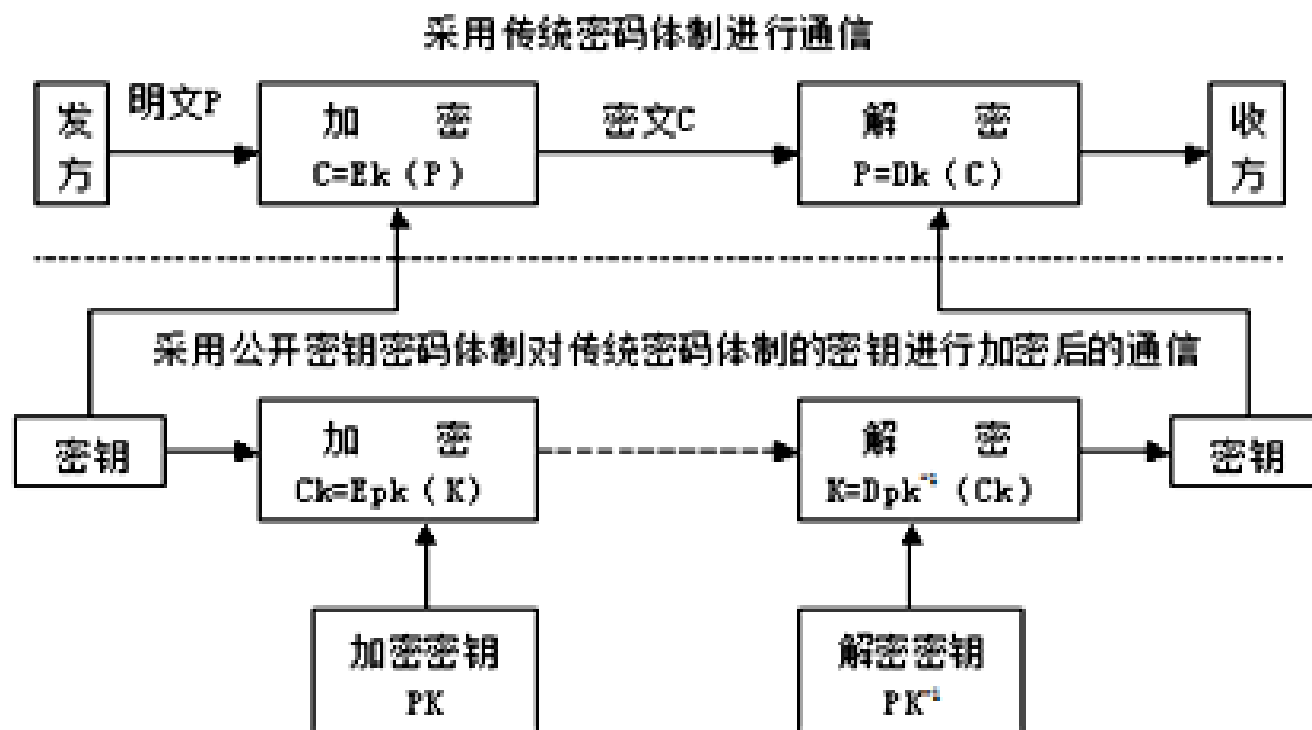
■ 缺点:

- **加解密速度慢**: 与对称加密相比, 非对称加密的加解密过程较为复杂, 处理速度较慢, 不适合大量数据加密;
- **计算资源需求高**: 非对称加密的计算量较大, 需要更多的计算资源。

5.1.2 密码学相关概念

■ 密码体制的分类

- **混合密码体制**: 混合密码体制是对称密码体制和非对称密码体制结合而成。



5.1.2 密码学相关概念

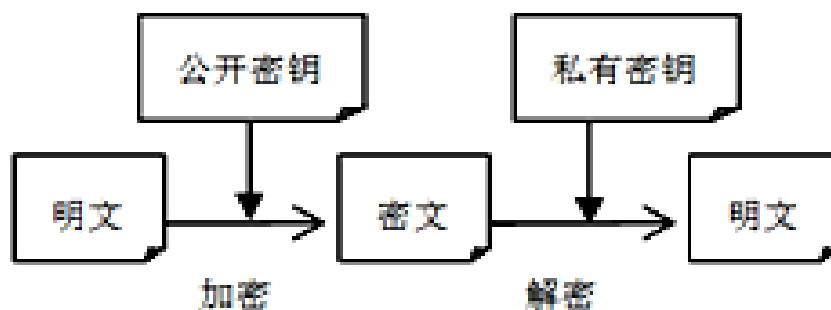
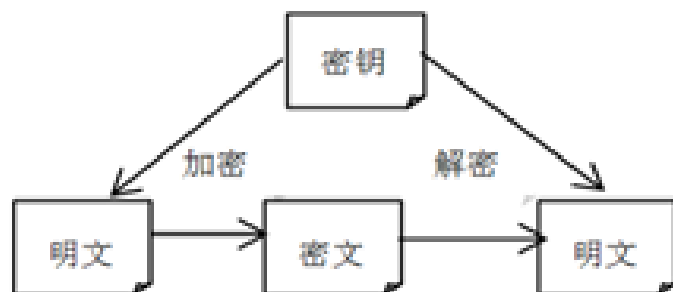
- 按照密码体制所处的时代，**密码体制**又可以**划分**为：
 - **古典密码体制**。指在计算机出现之前所涉及的密码体制。这类密码一般直接对明文采用**置换**和**代换**操作，运算较为简单，安全性差。
 - 包括：单表古典密码体制（如Caesar体制）、多表古典密码体制（如Playfair体制）。

a	b	c	d	e	f	g	h	i	j	k	l	m
d	e	f	g	h	i	j	k	l	m	n	o	p
n	o	p	q	r	s	t	u	v	w	x	y	z
p	r	s	t	u	v	w	x	y	z	a	b	c

$$P = \begin{pmatrix} c & i & p & h & e \\ r & a & b & d & f \\ g & k & l & m & n \\ o & q & s & t & u \\ v & w & x & y & z \end{pmatrix}$$

5.1.2 密码学相关概念

- **现代密码体制**。计算机出现后产生的密码体制，使用计算机加密运算较为复杂，破译难度大。
- 包括：对称密钥体制和非对称密钥体制。
- 对称密钥体制包括：分组密码、流密码等。
- 非对称密码体制代表：**RSA**算法。



5.1.2 密码学相关概念

■ 安全的密码体制应具有的性质：

- 从密文恢复明文应该很难，即使分析者知道明文空间，如明文是英语。
- 从密文计算出明文部分信息应该很难。
- 从密文探测出简单却有用的事实应该很难，如相同的信息被发送了两次。
- 上述三点保密程度、攻击目标、实现难度不同。



5.1.2 密码学相关概念

■ 密码体制安全性评价

■ 无条件安全性

如果一个密码体制**满足条件**：无论有多少可用的密文，都不足以确定密文所对应的明文，则称该密码体制是无条件安全的。

■ 计算安全性

■ 要求降低，更关心在**计算复杂性上不可破译**。

■ 计算复杂性不可破译的密码体制。必须**满足3个标准**：

■ 1) 破译密码的**代价**超出密文信息的价值；

■ 2) 破译密码**复杂度**超出了攻击者现有的计算能力；

■ 3) 破译密码**时间**超过了密文信息的有效生命期；

5.1.2 密码学相关概念

■ 密码体制安全性评价

- 一次一密——一种实现无条件安全性的途径
- 指在密码学中使用一次性密码本（One-Time Pad）进行加密和解密的技术。一次性密码本是一种理论上安全的加密方案，最早由美国密码学家沃纳·韦纳斯提出。
- 特点：密钥与明文等长、完全随机、只使用一次，并且发送者和接收者在事先共享同一密钥。
- 一次一密（One-Time Pad）被认为是在密码学中**具有绝对安全性的加密方案**，前提是严格满足了一次性密码本的要求。在理论上，如果一次性密码本被正确地生成、使用和保护，它提供了信息论上的完美保密性，即无论攻击者使用何种算法或计算资源，都无法从密文中获得有关明文的任何信息。

5.1.2 密码学相关概念

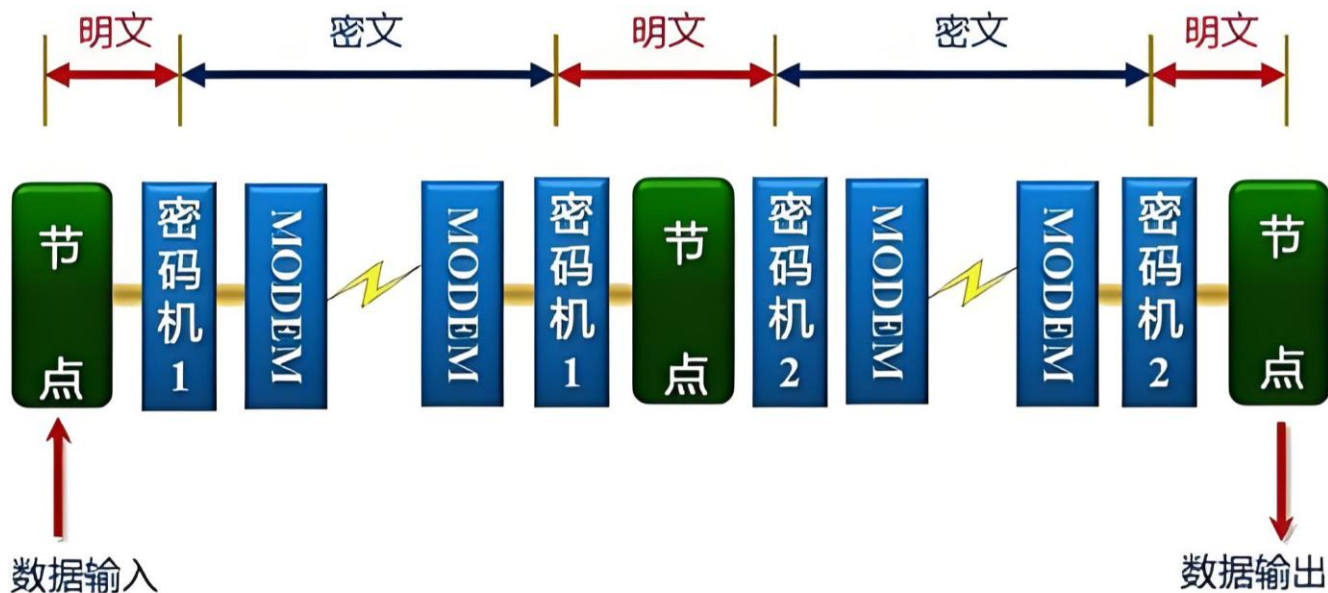
■ 密码体制安全性评价

- 一次一密实际应用中的困难：
- 密钥生成：一次性密码本要求生成完全随机。如果密钥生成不是真正随机的，就可能出现密钥的可推测性和重用，从而导致安全性受损。
- 密钥传输：一次性密码本要求发送者和接收者在事先共享相同的密钥，而且密钥的传输必须是安全的。如果密钥在传输过程中被拦截、篡改或泄露，密码的安全性将受到威胁。
- 密钥使用：一次性密码本要求每个密钥只使用一次，不可重复使用。如果密钥被重复使用，攻击者可能通过对比不同的密文来分析信息，并推导出密钥和明文的关系。
- 完整性和认证：一次性密码本只提供机密性保护，不能提供完整性验证和认证。这意味着攻击者可能篡改或替换密文而不被检测到。

5.1.3 数据及网络加密方式

■ 数据传输加密

- **链路加密**：对网络上传输**数据报文的每一位进行加密**，链路两端都用加密设备进行加密，保障整个通信链路传输安全。不考虑信源和信宿，只考虑**相邻节点间链路**上所传输数据的安全。
- **特点及应用**：只对传输链路中的数据加密，不对网络节点内数据加密，中间节点中数据报文以明文出现。对用户来说比较容易，**使用的密钥少**。目前，一般网络传输安全主要采用此方式。



5.1.3 数据及网络加密方式

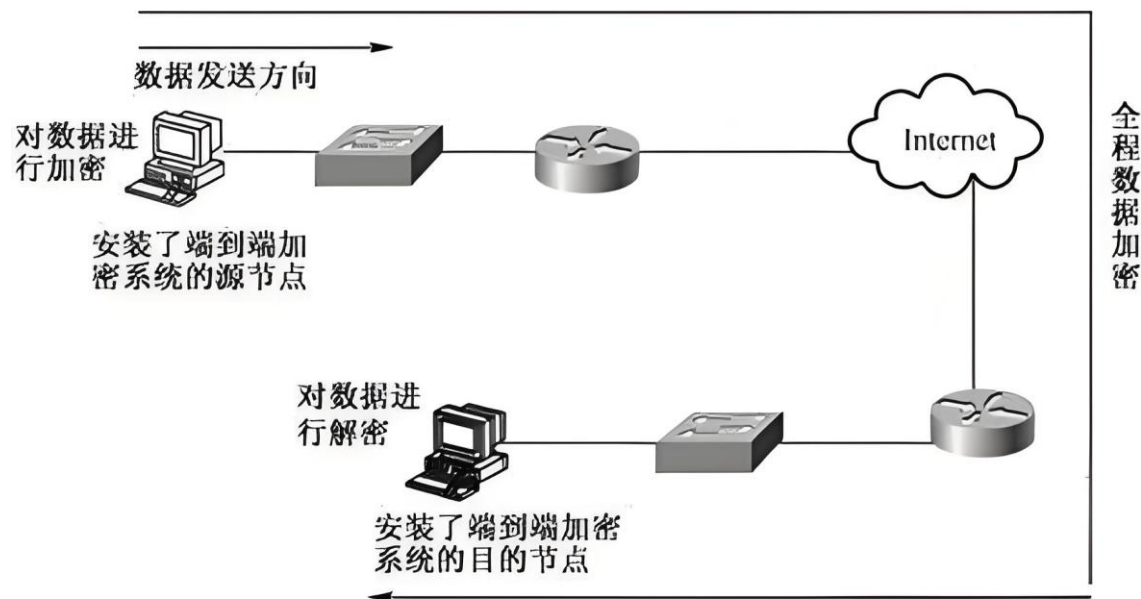
■ 数据传输加密

- **节点对节点加密**：在节点处采用一个与节点机相连的密码装置，密文在该装置中被解密并重新加密，明文不通过节点机，避免了链路加密节点处易受攻击的缺点。**仅对报文加密，不对报头加密**。
 - 。
- **缺点**：需要公共网络提供者配合，修改其交换节点，增加安全单元或保护装置；同时，节点加密要求报头和路由信息以明文形式传输，以便中间节点能得到如何处理消息的信息，也容易收到攻击。

5.1.3 数据及网络加密方式

■ 数据传输加密

- **端对端加密**：也称面向协议加密方式，是为数据从一端到另一端提供的加密方式。除报头外，报文均以密文形式贯穿于整个传输过程，也提供了一定程度的认证功能。
- 优点：每个用户可以有自己的密钥，不需额外增加专用加密设备。
- 缺点：每个系统都需自己的密钥管理软件，自己完成加密工作。



5.1.3 数据及网络加密方式

■ 数据传输加密

- 链路加密
- 节点对节点加密
- 端对端加密

■ 三种加密方式比较：

- 链路加密目的是保护链路两端网络设备间的通信安全；
- 节点加密目的是对源结点到目的结点之间的信息传输提供保护；
- 端到端加密目的是对源端用户到目的端用户应用系统通信提供保护。
- 链路加密和端对端加密方式的区别：链路加密方式是对整个链路的传输采取保护措施，而端对端方式则是对整个网络系统采取保护措施。端对端加密方式是未来发展主要方向。对于重要的特殊机密信息，可以采用将二者结合的加密方式。

5.1.3 数据及网络加密方式

■ 数据存储加密：

- 利用系统本身的加密功能加密
- 利用密码加密法
- 通过密钥加密



■ 数据存储加密方法比较：

- 利用系统本身的加密功能加密，**特点**是加密方式对系统的依赖性强，离开系统会出现无法读取现象；
- 密码加密法是读取时加密，没有对文件加密；通过密匙加密；是对文件整体加密。

5.1.3 数据及网络加密方式

讨论思考：

- (1)什么是密码学？密码体制有哪些？
- (2)数据加密技术主要分为哪两种加密方式？
- (3)对传输中的数据流进行加密方式有哪些？其特点？
- (4)数据加密技术在未来网络安全技术中的作用和地位？



使用密码技术不仅可以保证信息的（ ），而且可以保证信息的完整性和准确性，防止信息被篡改、伪造和假冒。

- ☒ A 机密性
- ☐ B 抗攻击性
- ☐ C 网络服务正确性
- ☐ D 控制安全性

网络加密常用的方法有链路加密、节点到节点加密和（ ）加密3种。

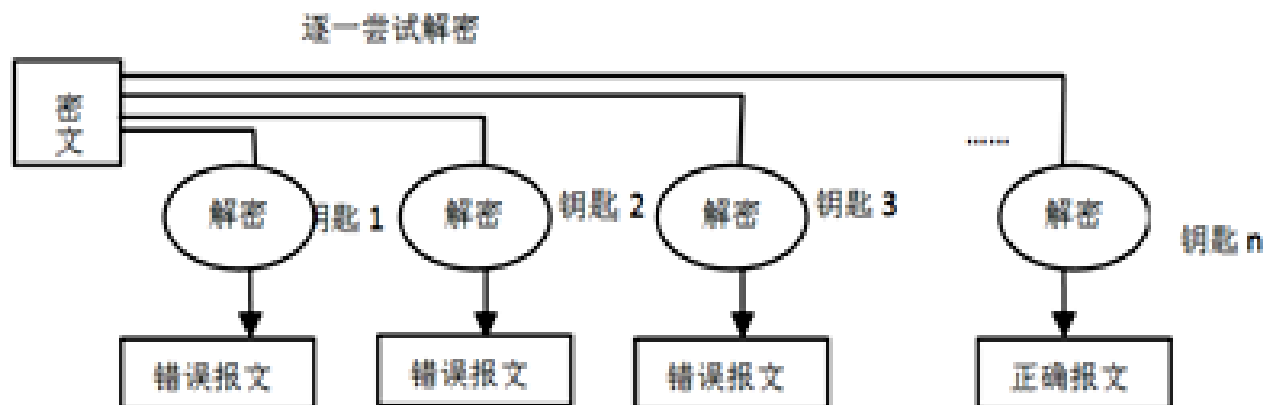
- ☐ A 系统
- ☒ B 端对端
- ☐ C 信息
- ☐ D 网站

5.2 密码破译与密钥管理

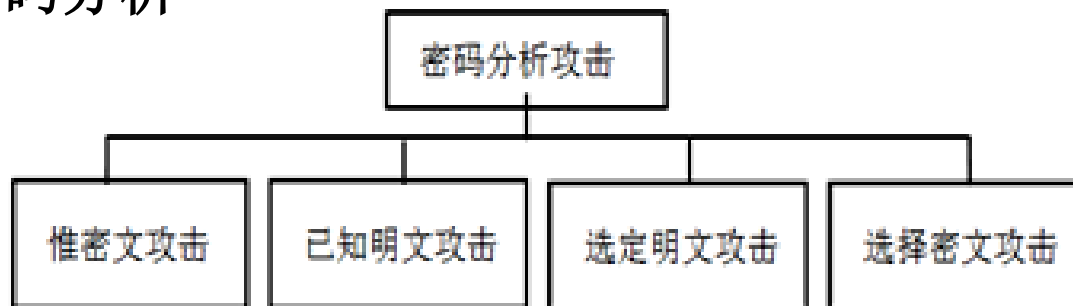
■ 密码破译方法

■ 穷举搜索密钥攻击

■ 例如，DES加密的密钥为56位。



■ 密码分析



5.2 密码破译与密钥管理

■ 1) 唯密文攻击 (Ciphertext-only attack)

- 已知加密算法，掌握部分要解密的密文，通过对这些密文进行分析得出明文或密钥。
- 唯密文破解**最容易防范**，因为攻击者拥有的信息量最少。但是在很多情况下，分析者可以得到更多的信息。如捕获到一段或更多的明文信息及相应的密文，也可能知道某段明文信息的格式。
- COA攻击者掌握的内容：截获的部分密文、加密算法。
- 例如，唯密文频率攻击。



5.2 密码破译与密钥管理

【案例 5-4】 电子邮件信头的格式总是固定的，如果加密电子邮件，必然有一段密文对应于信头。还有电子金融消息往往有标准化的文件头或者标志等。拥有这些知识的分析者就可以从转换明文的方法入手来推导出密文。

2) 已知明文攻击 (Known-plaintext attack)

- 已知加密算法，掌握了一段明文和对应的密文。**目的**是发现加密的钥匙。在实际使用中，获得与某些密文所对应的明文是可能的。
- KPA攻击者掌握的内容：截获的部分密文、加密算法、已知的部分明文以及对应的密文。

5.2 密码破译与密钥管理

- 3) 唯选定明文攻击 (Chosen-plaintext attack)
 - 已知加密算法，设法让对手加密一段选定的明文，并获得加密后的密文。**目的**是确定加密密钥。**差别比较分析法**也是其中一种，设法让对手加密一组相似却差别细微的明文，然后比较加密后结果，从而获得加密的钥匙。
 - CPA攻击者掌握的内容：截获的部分密文、加密算法、攻击者选择的明文以及对应的密文。

5.2 密码破译与密钥管理

- 4) 选择密文攻击 (Chosen-ciphertext attack)
 - 选择密文攻击指事先任意搜集一定数量的密文，让这些密文通过被攻击的加密算法解密，通过未知的密钥获得解密后的明文。在密码分析技术中很少用到。
 - CCA攻击者掌握的内容：截获的部分密文、加密算法、选择的密文以及对应的明文。

5.2 密码破译与密钥管理

- 四种攻击的攻击强度与实现难度
- 攻击强度：选择密文攻击 > 选择明文攻击 > 已知明文攻击 > 唯密文攻击
- 实现难度：选择密文攻击 < 选择明文攻击 < 已知明文攻击 < 唯密文攻击
- 攻击强度和实现难度之间存在一种反比关系：攻击越强，通常实现起来就越困难；反之亦然。现代加密算法的设计旨在抵抗所有这些攻击，尤其是 CCA 和 CPA，这使得在实际操作中成功破解它们变得极其困难。

5.2 密码破译与密钥管理

- 使用一个简单的偏移凯撒密码来加密和解密信息。假设我们的加密算法是将每个字母向后移动3个位置。
- 假设的明文信息: "Hello, World!"
- 加密算法: 每个字母向后移动3个位置
- 密文: 加密后的信息将是 "Khoor, Zruog!"

(1) 唯密文攻击

攻击者仅知道密文 "Khoor, Zruog!", 但不知道任何明文或加密算法的细节。在这种情况下, 攻击者必须依靠频率分析或猜测加密算法和偏移量。

对于 Caesar Cipher 来说, 这可以通过尝试所有 26 种可能的偏移量来完成。

5.2 密码破译与密钥管理

- 使用一个简单的偏移凯撒密码来加密和解密信息。假设我们的加密算法是将每个字母向后移动3个位置。
- 假设的明文信息: "Hello, World!"
- 加密算法: 每个字母向后移动3个位置
- 密文: 加密后的信息将是 "Khoor, Zruog!"

(2) 已知明文攻击

攻击者知道部分明文和对应的密文。例如, 攻击者知道明文 "Hello" 和密文 "Khoor"。这使攻击者能够轻易地推断出偏移量为3, 然后可以解密整个信息。

5.2 密码破译与密钥管理

- 使用一个简单的偏移凯撒密码来加密和解密信息。假设我们的加密算法是将每个字母向后移动3个位置。
- 假设的明文信息: "Hello, World!"
- 加密算法: 每个字母向后移动3个位置
- 密文: 加密后的信息将是 "Khoor, Zruog!"

(3) 选择明文攻击

攻击者可以自由选择明文并获得其密文。假设攻击者选择明文 "ABC" 并获得密文 "DEF"。由于这是一个简单的 Caesar Cipher, 攻击者立即知道偏移量为3, 因此可以解密任何其他密文。

5.2 密码破译与密钥管理

- 使用一个简单的偏移凯撒密码来加密和解密信息。假设我们的加密算法是将每个字母向后移动3个位置。
- 假设的明文信息: "Hello, World!"
- 加密算法: 每个字母向后移动3个位置
- 密文: 加密后的信息将是 "Khoor, Zruog!"

(4) 选择密文攻击

攻击者可以自由选择密文并获得其明文。如果攻击者有 "Khoor" 的密文, 他可以请求解密并得到明文 "Hello"。通过这种方式, 攻击者可以测试不同的密文片段, 找出加密算法的规律。

5.2 密码破译与密钥管理

■ 防止密码破译的措施

- **增强密码算法安全性。**通过增加密码算法的破译复杂程度,进行密码保护。例如增加密码系统的密钥长度,一般在其他条件相同的情况下,密钥越长破译越困难,而且加密系统也就越可靠。
- **使用动态会话密钥。**每次会话所使用的密钥不相同。
- **定期更换会话密钥。**



5.2 密码破译与密钥管理

■ 密钥管理

- 是指对所用密钥生命周期的全过程实施的安全保密管理。
- 影响到整个系统的安全，同时也涉及系统的可靠性、有效性和经济性。
- 包括密钥的产生、存储、分配、使用和销毁等一系列技术问题。
- **主要任务**在公用数据网上安全地传递密钥不被窃取。
- 主要有两种**网络密钥管理方法**：**KDC(Key Distribution Center)**和**Diffie-Hellman**。

5.2 密码破译与密钥管理

■ 密钥管理

- KDC使用可信第三方来验证通信双方的真实性，产生会话密钥，并通过数字签名等手段分配密钥。
- Diffie-Hellman则不需KDC，通信发起方产生通信会话的私用密钥，并通过数字签名或零知识证明等方式安全传递通信密钥。
- 网络密钥主要有三种：
 - 会话密钥(Session Key)
 - 基本密钥(Basic Key)
 - 主密钥(Master Key)。

5.2 密码破译与密钥管理

■ 零知识证明

- 是指一种密码学工具，允许互不信任的通信双方之间证明某个命题的有效性，同时不泄露任何额外信息。
- 由莎菲·戈德瓦塞尔、S.Micali及C.Rackoff在20世纪80年代初提出的。它指的是证明者能够在不向验证者提供任何有用的信息的情况下，使验证者相信某个论断是正确的。
- 举例：
 - 1、有一个缺口环形的长廊，出口和入口距离非常近（在目距之内），但走廊中间某处有一道只能用钥匙打开的门，A要向B证明自己拥有该门的钥匙。采用零知识证明，则B看着A从入口进入走廊，然后又从出口走出走廊，这时B没有得到任何关于这个钥匙的信息，但是完全可以证明A拥有钥匙。

5.2 密码破译与密钥管理

■ 零知识证明

- 2、A要向B证明自己拥有某个房间的钥匙，假设该房间只能用钥匙打开锁，而其他任何方法都打不开。这时有2个方法：
 - ①A把钥匙出示给B，B用这把钥匙打开该房间的锁，从而证明A拥有该房间的正确钥匙。
 - ②B确定该房间内有某一物体，A用自己拥有的钥匙打开该房间的门，然后把物体拿出来出示给B，从而证明自己确实拥有该房间的钥匙。

5.2 密码破译与密钥管理

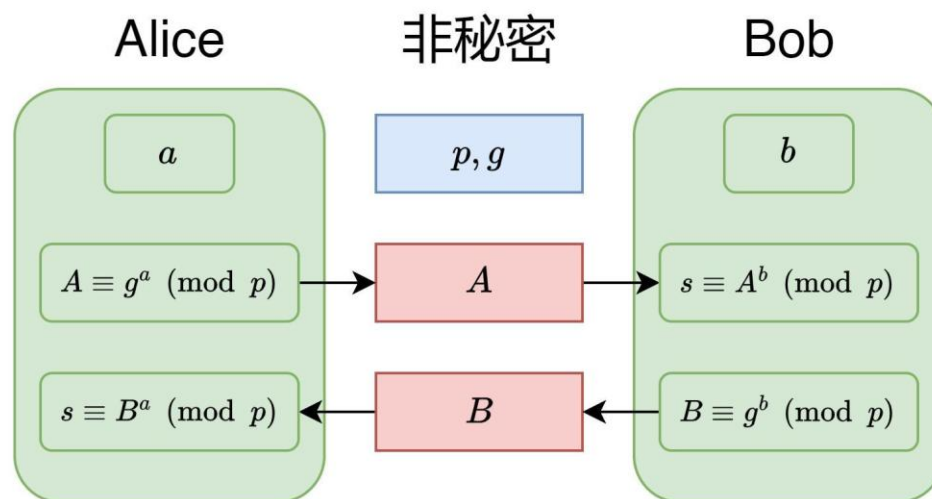
■ 零知识证明

- 3、A拥有B的公钥，A没有见过B，而B见过A的照片，偶然一天两个人见面了，B认出了A，但A不能确定面前的人是否是B，这时B要向A证明自己是B，也有2个方法。
- ① B把自己的私钥给A，A用公钥对某个数据加密，然后用B的私钥解密，如果正确，则证明对方确实是B。
- ② A给出一个随机值，并使用B的公钥对其加密，然后将加密后的数据交给B，B用自己的私钥解密并展示给A，如果与A给出的随机值相同，则证明对方是B。

5.2 密码破译与密钥管理

■ *Diffie-Hellman算法 ▶

- 仅当需要时才生成密钥，减少了因密钥存储期长而使遭受攻击的机会；除对全局参数的约定外，密钥交换不需要事先存在的基础结构。



非对称密码分配密钥方法归纳为：公开发布；公开访问目录；公钥授权；公钥证书。

5.2 密码破译与密钥管理

■ 密钥分配协定机制

- 密钥分配：系统中的一个成员先选择一个秘密密钥，然后将它传送另一个成员/别的成员。**密钥协定**是一种协议，是通过两个/多个成员在一个公开的信道上通信联络建立一个秘密密钥。

■ 理想的密钥分配协议应满足两个条件：

- 传输量和存储量都比较小；
- 每对用户U和V都能**独立**地计算一个**秘密**密钥K。



5.2 密码破译与密钥管理

讨论思考

- (1) 对称密码体制破译的两种主要方法是什么？如何预防？
- (2) 什么是密钥管理？为什么要进行密钥管理？
- (3) 密钥管理包含内容是什么？主要密钥管理技术有哪些？



根据密码分析者破译时已具备的前提条件，通常人们将攻击类型分为4种：一是（ ），二是（ ），三是选择明文攻击，四是选择密文攻击。

- ☐ A 已知明文攻击、选择密文攻击
- ☐ B 选择明文攻击、已知明文攻击
- ☐ C 选择密文攻击、唯密文攻击
- ☒ D 唯密文攻击、已知明文攻击

5.3 实用密码技术概述

实用密码技术主要包括2种：

- 对称密码体制

- 古典对称密码
- 现代分组密码
- 现代流密码算法
- 现代散列算法

- 非对称密码体制

- 数字签名（综合应用散列算法和非对称密码体制）



古典对称密码

■ 1) 代换技术

■ **代换技术**是将明文中的每个元素（字母、比特、比特组合或字母组合）映射为另一个元素的技术，即明文的元素被其他元素所代替而形成密文。**常见的代换技术的古典对称密码**包括凯撒密码、单字母替换密码及Vigenere密码。

维吉尼亚



5.3 实用加密技术概述

- 凯撒密码是最古老的一种单表替代密码。（特殊的移位密码）
- 凯撒密码将字母表视为一个循环表，把明文中的字母用表中该字母后面第3个字母进行替代，作为密文。
- 即密文字母相对明文字母循环右移3为，所以也成“循环移位密码”。
- 如果让每个字母对应一个数值（a=0, b=1, ..., z=25），则该算法可以表示为：

$$c = E(p) = (p + k) \bmod 26$$

$$p = D(c) = (c - k) \bmod 26$$

其中，**p**：明文信息元素；**c**：密文信息元素；**k**：密钥（取值1~25）

【案例 5-6】 密文：R N R F Y N E M N

密钥：k = 5

明文：M I M A T I Z H I

特点：密钥k的取值范围小，即密钥空间小，难以抵御穷举密钥攻击，攻击者最多尝试25次，就一定能破译密码。

5.3 实用加密技术概述

- 为了加大凯撒密码的密钥空间，可采用单字母替代密码。
- 单字母替代密码是将密文字母的顺序打乱后，每条信息用一个字母表（给出从明文字母到密文字的映射）加密。

明文字母: a b c d e f g h i j k l m n o p q r s t u v w x y z

密文字母: o g r f c y s a l x u b z q t w d v e h j m k p n i

【案例 5-7】 明文: happynewyear

密文: aowwnqckncov

特点: 密钥空间大小为 $26!$ ，约为 4×10^{26} ，如每微秒尝试一个密钥，也需1010年才能穷举完所有密钥，计算上不可破解，能够抵御穷举密钥攻击。

5.3 实用加密技术概述

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

明文: ATTACKATDAWN

密钥: LEMONLEMONLE

密文: LXFOPVEFRNHR

用数字0-25代替字母A-Z, 维吉尼亚密码的加密文法可以写成同余的形式:

$$C_i \equiv P_i + K_i \pmod{26}$$

解密方法则能写成:

$$P_i \equiv C_i - K_i \pmod{26}$$

特点: 对多表密码的分析和攻击一般比单表密码难。即使只选择矩阵中任意 m 行, 其密钥长度也将是 26^m 。

Vigenere密码 (多表密码)

5.3 实用加密技术概述

■ 2) 置换技术

- **置换**是在不丢失信息的前提下,对明文中的元素进行重新排列,常见的置换古典密码有**矩阵置换**。
- **矩阵置换**:把明文中的**字母**按给定的顺序**安排**在一**矩阵**中,然后用另一种顺序**选出**矩阵的字母产生**密文**。



5.3 实用加密技术概述

【案例5-9】 明文 please send the data (空格为一个字符) 使用矩阵转置技术的密码。以下是 3 个在不同输出顺序下的密文例子。矩阵转置密码加密示意图如图 5-7 所示。矩形方阵上方的数字和字母串代表输出顺序的密钥 (说明: 密文的输出顺序, 数字按小到大, 字母串按字母顺序输出 (AEFRT))。明文: PLEASESENDTHEDATA.

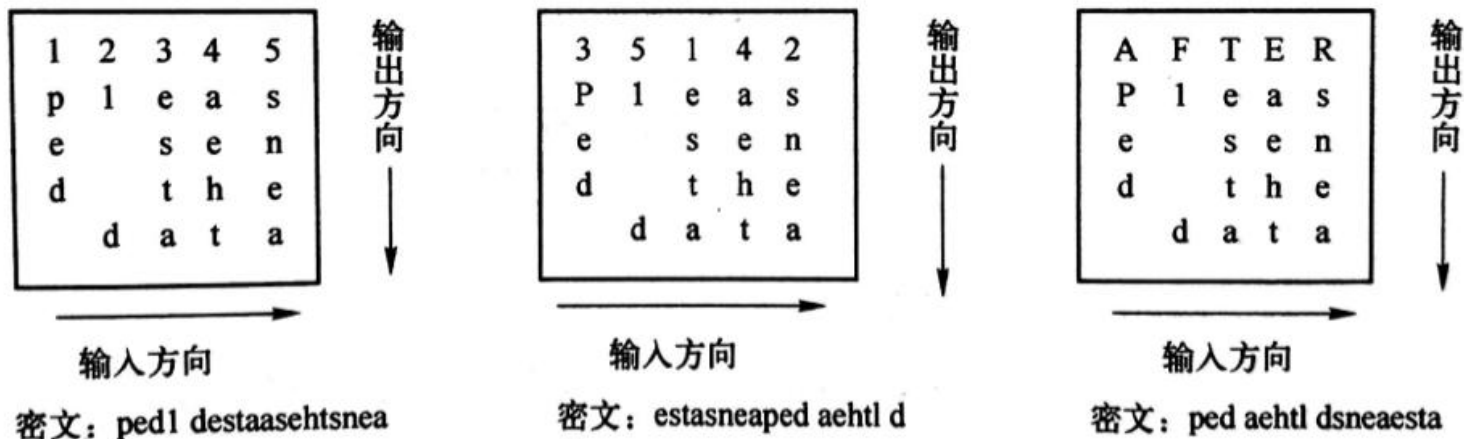


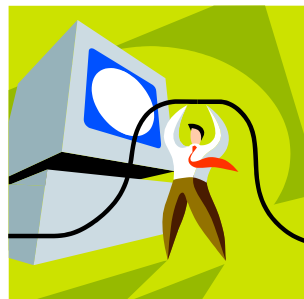
图 5-7 矩阵转置密码加密示意图

5.3 实用加密技术概述

2. 现代对称加密技术（分组密码和流密码）

如果在一个密码体系中，**加密密钥**和**解密密钥**相同，就称为**对称加密**。现代密码技术阶段**加密**和**解密算法**是**公开的**，数据的**安全性**完全**取决于**密钥的安全性，因此，对称加密体系中如果**密钥**丢失，数据将**不再安全**。

代表性的对称加密算法有**DES**（数学加密标准），**IDEA**（国际数据加密算法），**Rijndael**，**AES**，**RC4**算法等。



数据加密标准算法DES

- *最早得到最广泛应用的分组密码算法是**数据加密标准DES** (Data Encryption Standard)算法, 是由IBM公司在70年代发展起来的。
- DES于1976年11月被美国政府采用, 随后被美国国家标准局(现为国家标准和技术研究所NIST)承认, 并被采纳为联邦信息处理标准46 (FIPS PUB 46)。
- **DES算法**采用了64位的分组长度和56+8位的密钥长度。它将64位的比特输入经过16轮迭代变换得到64位比特的输出, 解密采用相同的步骤和相同的密钥。



5.3 实用加密技术概述

■ *1) DES

算法思想：DES算法将输入的明文分为64位的数据**分组**，使用64位的**密钥**进行**变换**，每个64位**明文分组**数据经过**初始置换**、16次**迭代**和**逆初始置换**3个主要阶段，最后**输出**得到64位**密文**。

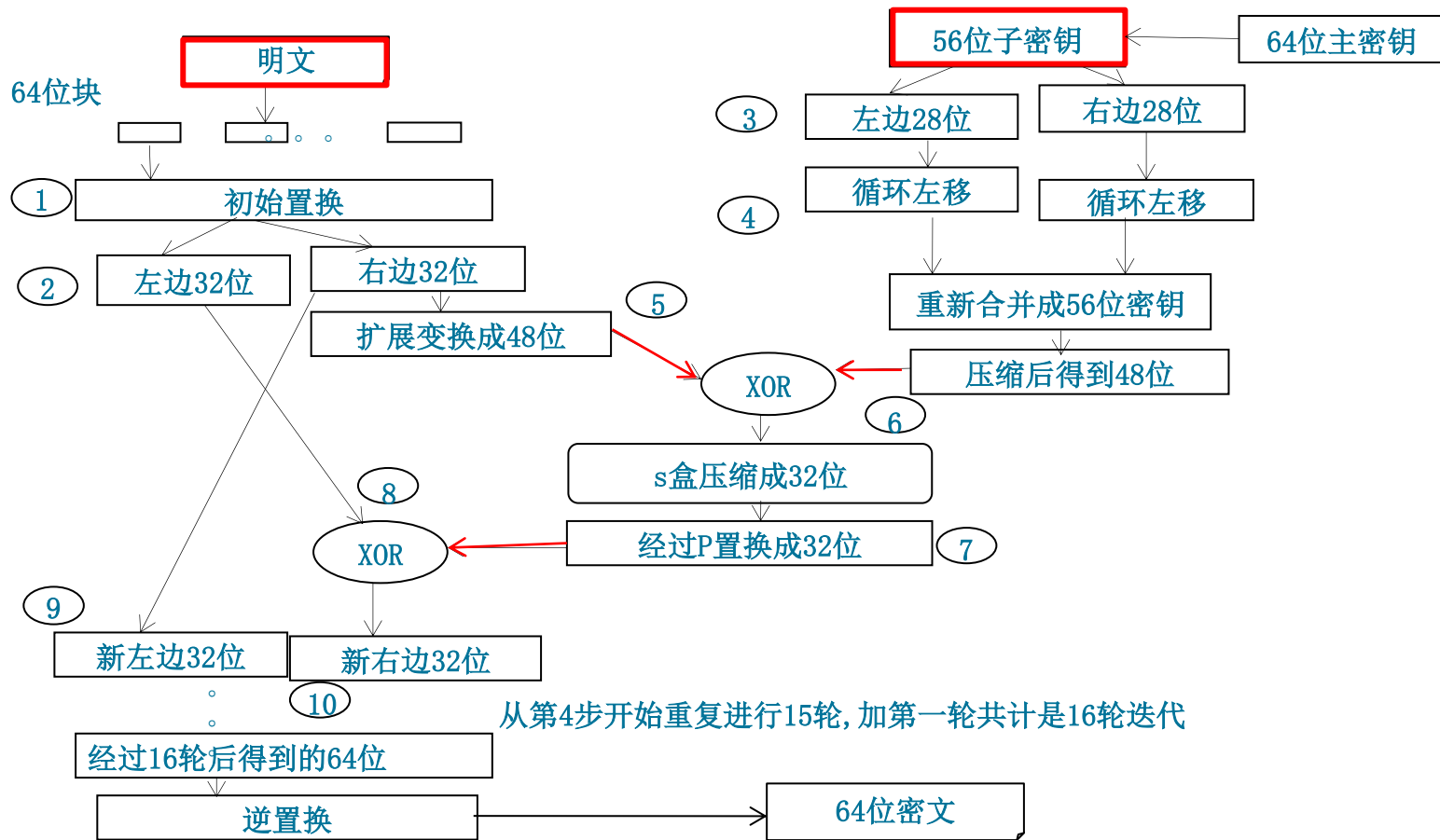


图5-7 DES加密算法

5.3 实用加密技术概述

■ DES初始置换和终止置换

<i>Initial Permutation</i>	<i>Final Permutation</i>
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25

5.3 实用加密技术概述

■ DES初始置换和终止置换

初始置换
置换表

58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5	63	55	47	39	31	23	15	7

原始数据

0	1	1	0	0	0	1	1	0	1	0	1	1	1	0	0
0	0	1	1	1	0	1	1	1	0	1	1	0	1	0	1
0	0	0	1	0	1	1	1	1	0	0	0	0	0	0	1
0	0	1	1	1	0	0	0	0	0	1	0	0	1	1	0

置换后

0	0	0	0	0	0	1	1	0	1	0	1	1	1	1	0
1	0	0	1	1	0	1	0	0	0	1	1	1	1	0	1
0	0	1	0	1	0	0	0	1	1	0	0	1	1	0	1
0	1	0	0	0	1	1	0	1	0	0	1	0	1	0	1

5.3 实用加密技术概述

■ DES初始置换和终止置换

例1: 当输入如下的一个十六进制数时, 请写出终止置换的输出。

0x0000 0080 0000 0002

解答:

0x0000 0080 0000 0002

终止置换表															
40	08	48	16	56	24	64	32								
39	07	47	15	55	23	63	31								
38	06	46	14	54	22	62	30								
37	05	45	13	53	21	61	29								
36	04	44	12	52	20	60	28								
35	03	43	11	51	19	59	27								
34	02	42	10	50	18	58	26								
33	01	41	09	49	17	57	25								

0x0002 0000 0000 0001

转换为二进制

0000000000000000
0000000010000000
0000000000000000
0000000000000010

终止置换

转换为十六进制

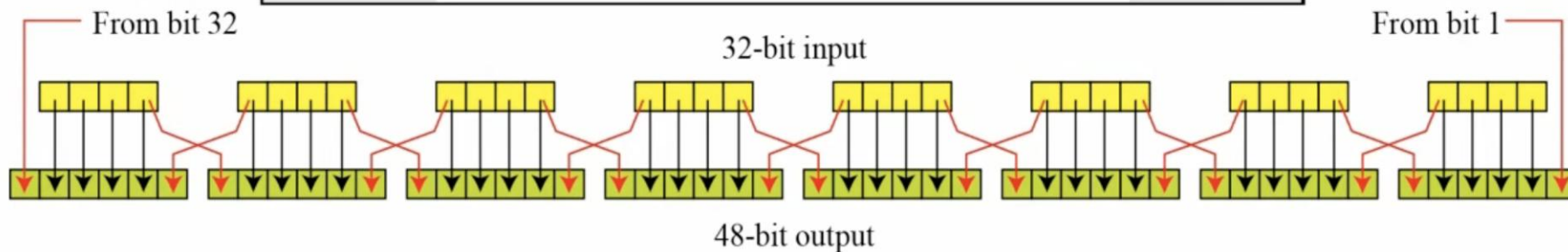
0000000000000010
0000000000000000
0000000000000000
0000000000000001

5.3 实用加密技术概述

■ DES扩展置换

扩展置换表

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01



5.3 实用加密技术概述

■ DES扩展置换

扩展置换
置换表

32	1	2	3	4	5	4	5	6	7	8	9
8	9	10	11	12	13	12	13	14	15	16	17
16	17	18	19	20	21	20	21	22	23	24	25
24	25	26	27	28	29	28	29	30	31	32	1

1 1 0 1

0 0 0 1

0 0 1 1

0 1 0 0

1 0 1 0 1 0 1 0 1 0 1 0

0 0 1 0

0 0 1 1

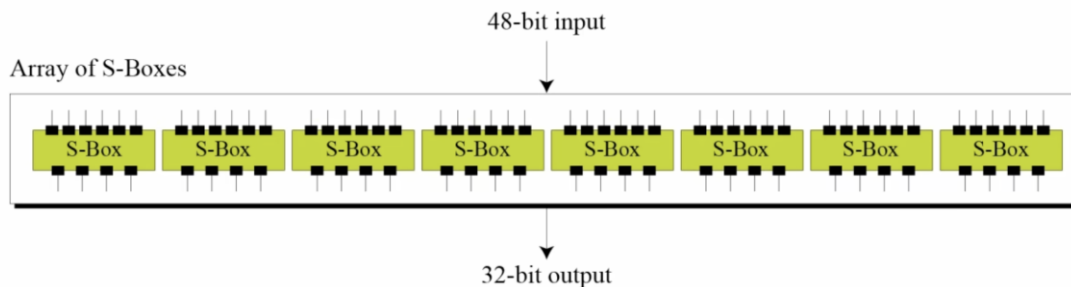
0 0 1 1

1 0 1 1

0 1 0 1 0 1 1 1 0 1 1 1

5.3 实用加密技术概述

■ DES的S盒压缩处理



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S3	0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S4	0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S5	0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S6	0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S7	0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S8	0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

5.3 实用加密技术概述

■ DES的S盒压缩处理

S盒压缩处理

原始数据:

1 1 1 1 1 1

头尾数据:

1 1

转化成十进制

3

即为行数

中间数据:

1 1 1 1

转化成十进制

15

即为列数

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

3 行
15 列

压缩结果: 13

转换成二进制

1101

5.3 实用加密技术概述

■ DES的P盒置换

P 盒置换表							
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

5.3 实用加密技术概述

■ DES密钥压缩置换

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

主密钥64位

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

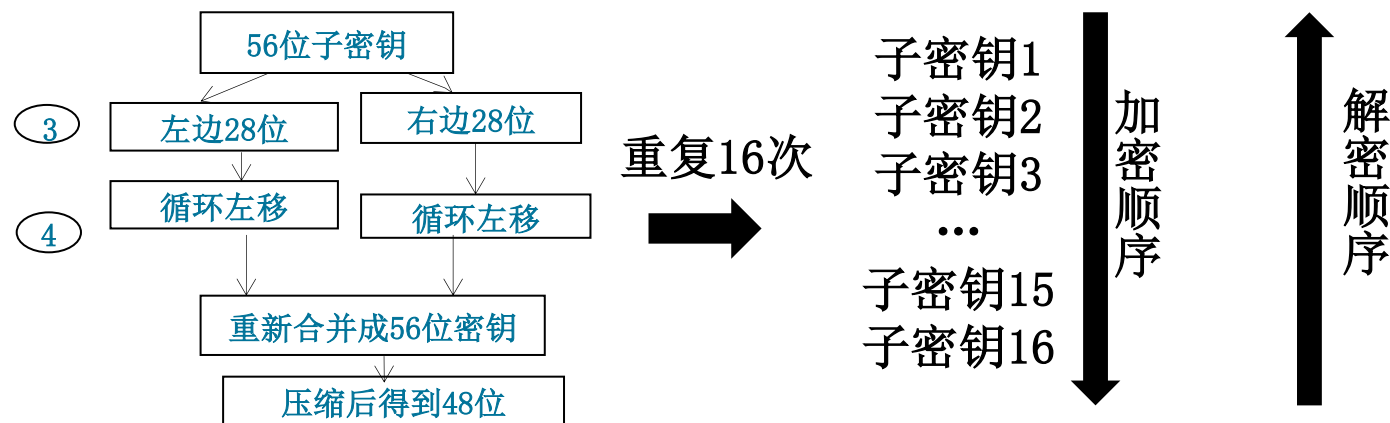
子密钥交换规则表
64位→56位

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

压缩置换规则表
56位→48位

5.3 实用加密技术概述

- DES解密过程
- DES由于是基于Feistel的，因此其加解密算法完全一样，只需将加密后的二进制密文作为输入，在进行一次加密即可得到解密结果，注意，解密时轮密钥应倒着使用。





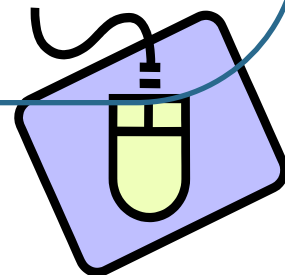
5.3 实用加密技术概述

- DES 综合运用了置换、代换技术。DES 用软件进行加、解密需要用很长时间，而用硬件加、解密速度非常快。DES 算法加密步骤和解密步骤是相同的，只是密钥输入顺序不同而已。因此在制造 DES 芯片时能节约门电路，容易达到标准化和通用性，适合用在硬件加密设备中。
- DES 正式公布后，世界各国的许多公司都推出了自己实现 DES 的软硬件产品，美国 NBS 至少已认可了 30 多种硬件和软件实现产品。硬件产品既有单片式的，也有单板式的；软件产品既有利于大中型机的，也有用于小型机和微型机的。
- DES 曾经是世界上最广泛的密码算法，但随着计算机运行速度的提高，56bit 的密钥长度已经不能够抵抗现有的穷举密钥攻击，DES 算法已经渐渐退出了历史舞台。

5.3 实用加密技术概述

- *2) 三重DES
- DES算法现在已经不能提供足够的安全性，因为其有效密钥只有56位。因此，后来又提出了三重DES（或称3DES），该方法的强度大约和112比特的密钥强度相当。
- 这种方法用两个密钥对明文进行三次运算。设两个密钥是K1和K2：
 - (1) 用密钥K1进行DES加密。
 - (2) 用K2对步骤1的结果进行DES解密。
 - (3) 用步骤2的结果使用密钥K1（可以使用新密钥K3）进行DES加密。

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$



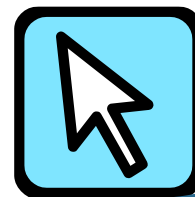
三重DES

- ***三重 (Triple DES)** 是DES的加强版。它能够使用多个密钥，对信息逐次作三次DES加密操作。3DES在使用DES算法三次，其中可以用到两组或者三组56比特长度密钥。
- 3DES有2个显著的优点。首先其密钥长度可以达到168比特，能克服 DES 面对的穷举攻击问题；其次，3DES 的底层加密算法与 DES 的加密算法相同，使得原有的加密设备能够得到升级。
- 最后，DES加密算法比其他加密算法受到分析的时间要长得多，相应地3DES对分析攻击有很强的免疫力。缺点是用软件实现该算法比较慢。



高级数据加密标准算法 AES

- *NIST在2001年发布了**高级加密标准AES**（Advanced Encryption Standard）。NIST从最终的五个候选者中选择Rijndael算法作为AES标准。
 - AES的**分组长度**128位,**密钥长度**可以为128、192或256位。
 - 目前最流行的版本是128比特密钥长度的AES-128, 其对128比特的消息块使用10轮迭代后得到密文。
 - 密钥长度每多64位, 迭代多2轮。
 - AES算法具有能抵抗所有的已知攻击, 平台通用性强, 运行速度快, 设计简单等特点。
 - 相比DES, AES的安全性更好, 但其加密步骤和解密步骤不同, 因而其硬件实现没有DES简单。



高级数据加密标准算法 AES

- **AES-128 加密过程**是在一个 4×4 的字节状态矩阵上进行的，矩阵中每个元素的大小就是明文中的一个字节，共组成了明文消息块的 128bit。
- AES-128 共需要10轮循环迭代，加密时除第 10轮外，1~9 轮循环迭代均包含4个步骤。第 10 轮加密循环迭代中省略了列混淆步骤。

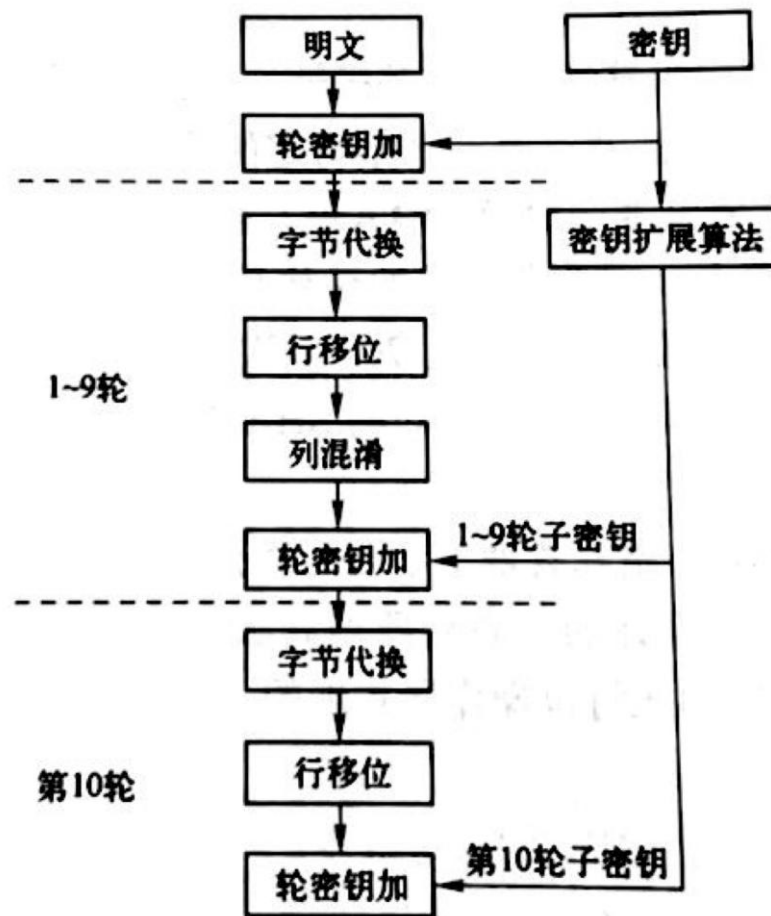
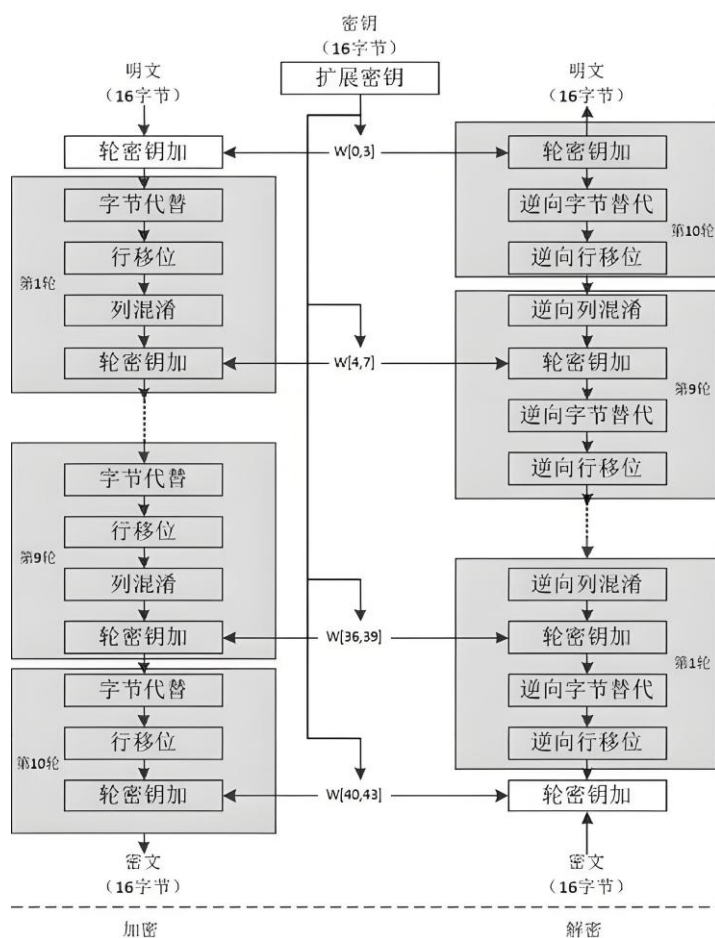


图 5-9 AES-128 加密过程

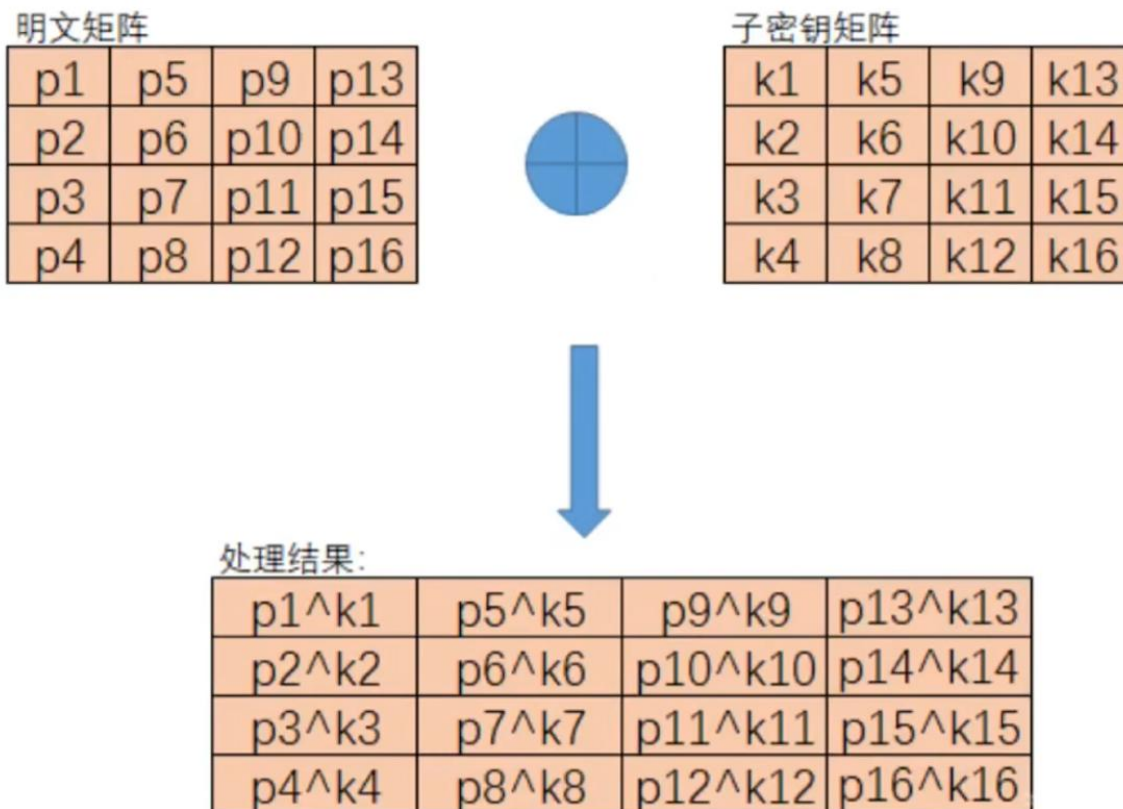
高级数据加密标准算法 AES

- AES-128 加密过程**是在一个 4×4 的字节状态矩阵上进行的，矩阵中每个元素的大小就是明文中的一个字节，共组成了明文消息块的 128bit。AES-128 共需要10轮循环迭代。



高级数据加密标准算法 AES

- 1) **轮密钥加**: 状态中的每一个字节都与该轮子密钥做异或运算。



高级数据加密标准算法 AES

2) **字节代换**: 通过一个非线性的替换函数, 用查表的方式把每个字节替换成对应的字节。

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

假设这是一个完成初始变换的矩阵

d4	e0	b8	1e
27	bf	b4	41
11	98	5d	52
ae	f1	e5	30

全部
替换后

以矩阵的第一个元素为例, 19代表第1行第9列, 去S-BOX中找到这个元素对原矩阵进行替换

hex	y															
	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

S-BOX

高级数据加密标准算法 AES

- 3) 行移位：将矩阵中的每个横列进行循环式移位。

输入数据

p1	p5	p9	p13
p2	p6	p10	p14
p3	p7	p11	p15
p4	p8	p12	p16

行位移

输出结果

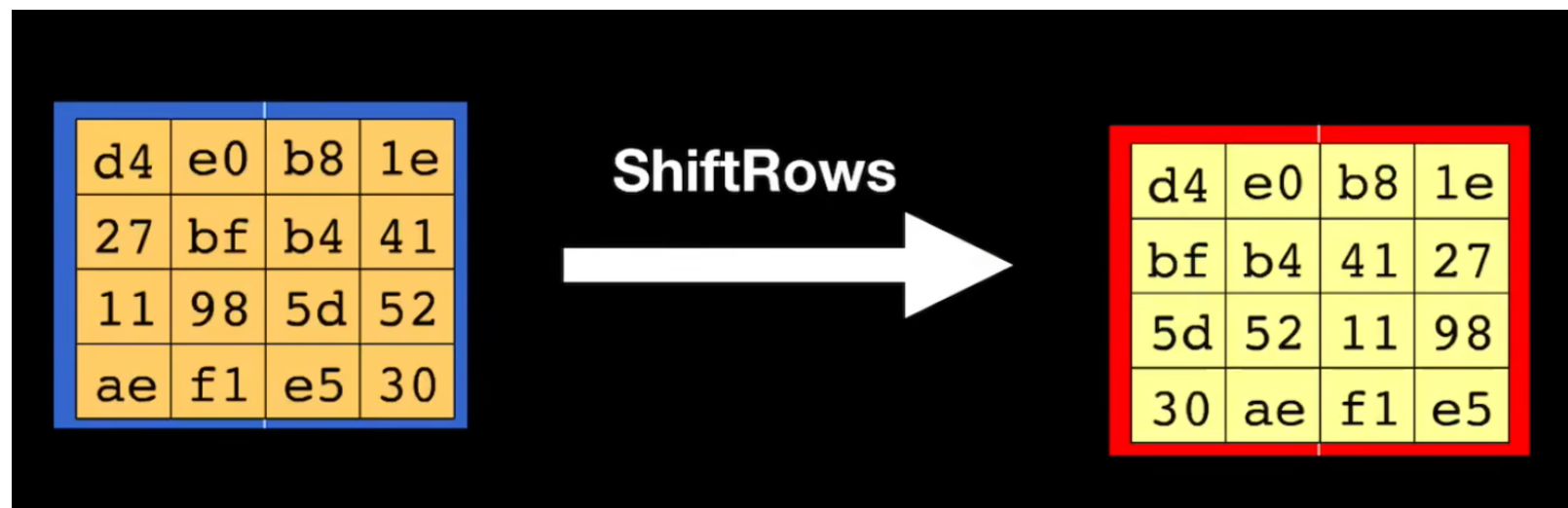
p1	p5	p9	p13
p6	p10	p14	p2
p11	p15	p3	p7
p16	p4	p8	p12

保持第一行不变

第二行向左移动1个字节

第三行向左移动2个字节

第四行向左移动3个字节



高级数据加密标准算法 AES

■ 4)列混淆：使用线性变换来混合每列的4个字节。

将输入的4*4的矩阵左乘一个给定的4*4矩阵

正矩阵

0x02	0x03	0x01	0x01
0x01	0x02	0x03	0x01
0x01	0x01	0x02	0x03
0x03	0x01	0x01	0x02

输入数据

p1	p5	p9	p13
p2	p6	p10	p14
p3	p7	p11	p15
p4	p8	p12	p16



02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

MixColumns



04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

高级数据加密标准算法 AES

■ 轮密钥生成：拓展时，使用如下规则

- 如果i不是4的倍数，那么第i列由如下等式确定：

$$a[i] = a[i-4] \oplus a[i-1]$$

- 如果i是4的倍数，那么第i列由如下等式确定：

$$a[i] = a[i-4] \oplus T(a[i-1])$$

2b	28	ab	09	a0	88	23	2a	f2	7a	23	73	3d	47	1e	6d
7e	ae	f7	cf	fa	54	a3	6c	c2	96	a3	59	80	16	23	7a
15	d2	15	4f	fe	2c	39	76	95	b9	39	f6	47	fe	7e	88
16	a6	88	3c	17	b1	39	05	f2	43	39	7f	7d	3e	44	3b

Cipher Key

Round key 1

Round key 2

Round key 3

...

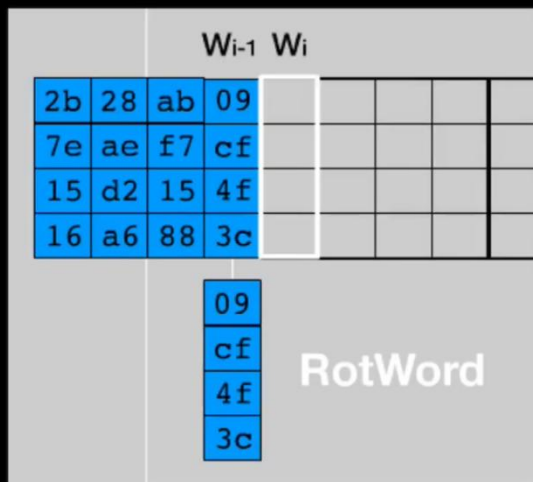
d0	c9	e1	b6
14	ee	3f	63
f9	25	0c	0c
a8	89	c8	a6

Round key 10

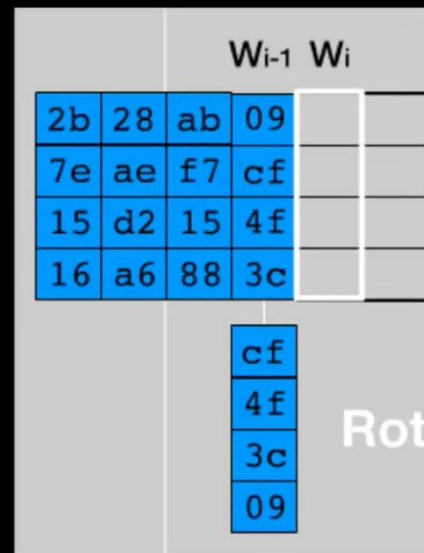
高级数据加密标准算法 AES

T函数——①字循环：

a.字循环：将1个字中的4个字节循环左移1个字节。即将输入字[b0, b1, b2, b3]变换成[b1,b2,b3,b0]。



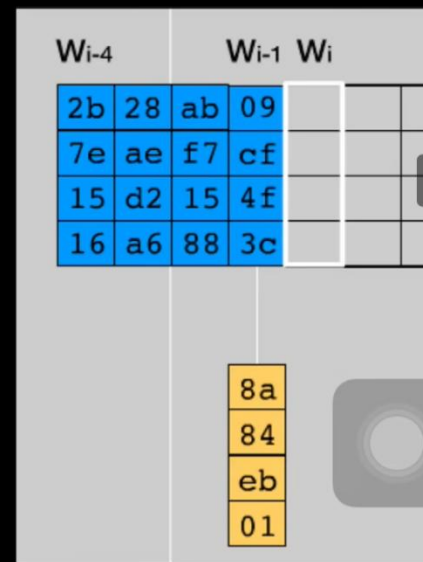
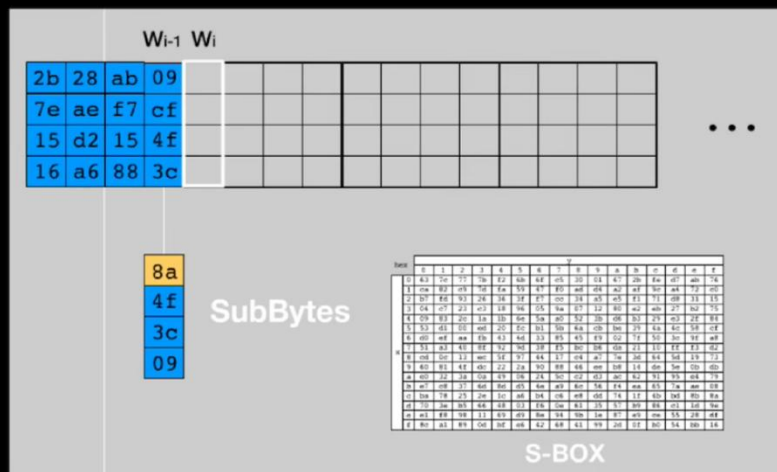
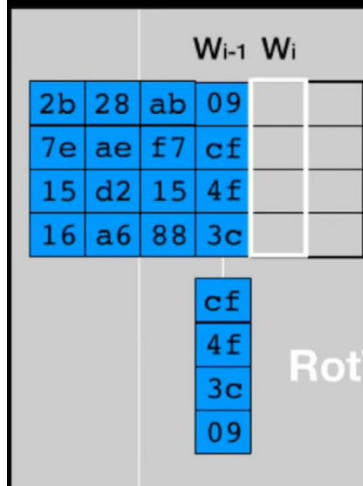
a.字循环



高级数据加密标准算法 AES

T函数——②字节代换:

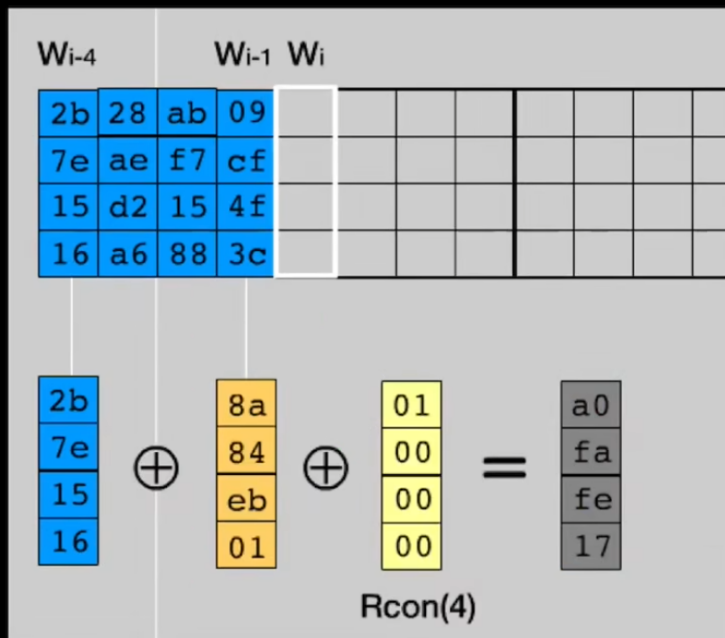
b.字节代换：对字循环的结果使用S盒进行字节代换。



高级数据加密标准算法 AES

T函数——③轮常量异或：

c.轮常量异或：将前两步的结果同轮常量Rcon[j]进行异或，其中j表示轮数。



2b	28	ab	09	a0		
7e	ae	f7	cf	fa		
15	d2	15	4f	fe		
16	a6	88	3c	17		

分组密码设计思路

- DES和AES是两种经典的分组加密算法，它们的设计思路体现了密码学从Feistel结构到SPN结构（Substitution-Permutation Network）的演进。

特性	Feistel结构	SPN结构
加解密结构	相同（自对偶）	不同
轮函数要求	不需要可逆	S盒必须可逆
扩散速度	较慢（需多轮）	较快（一轮全扩散）
并行性	较低	较高
实现复杂度	简单	稍复杂
代表算法	DES	AES

分组密码设计思路

- 代换-置换（Substitution-Permutation，简称S-P结构）是分组密码设计的核心范式，其根本目的是实现密码学的两大核心原则：**混淆（Confusion）** 和 **扩散（Diffusion）**。

原则	目的	实现方式	作用
混淆	使密文与密钥的关系尽可能复杂，隐藏密钥信息	代换 (Substitution) — S盒	混淆 提供非线性 抵抗已知攻击（差分分析、线性分析）
扩散	使明文的每一位影响密文的多个统计特征	置换 (Permutation) — P盒	扩散 打乱统计规律

分组密码设计思路

■ 为什么必须两者结合？

仅用代换	仅用置换	两者结合
局部混淆，无扩散	位置打乱，无非线性	全局混淆+全局扩散
易被统计分析	易被代数攻击	抵抗多种攻击

■ 雪崩效应 (Avalanche Effect)

■ 理想情况下，改变输入1比特，输出约**50%比特改变**（最大熵）：

- 明文： 0000 0000 0000 0000 ... (64位)
- 密文A: 1110 0101 1011 0010 ...
- 明文： 0000 0000 0000 0001 ... (仅最后1位改变)
- 密文B: 0011 1100 0101 1110 ... (约32位不同)

■ 消除明文统计信息、破坏差分模式。

■ 代换-置换的本质目的：在密钥控制下，将明文通过**可逆的非线性变换**，转化为**统计独立、与密钥复杂关联**的密文。

分组密码运行模式

- 分组密码需要不同运行模式，核心原因在于：基础的分组加密算法（如AES、DES）本身只能安全地加密固定长度的数据块，而现实世界的的数据千差万别，直接裸用会带来严重的**安全漏洞**：
 - 问题1：确定性输出
 - 攻击者一眼看出明文重复模式
 - 可替换、重排密文块而不被发现
 - 无法加密结构化数据
 - 问题2：长度限制
 - AES固定处理128位（16字节），实际数据可能是任意长度：5字节、1000字节、1GB...
 - 问题3：缺乏完整性保护
 - 加密只保证机密性，不保证完整性
 - 攻击者可篡改密文，解密后得到错误但“合法”的明文

分组密码运行模式

- 对大块数据进行加密时，需要使用分组密码相应的运行模型，**常见的分组密码运行模式**包括：

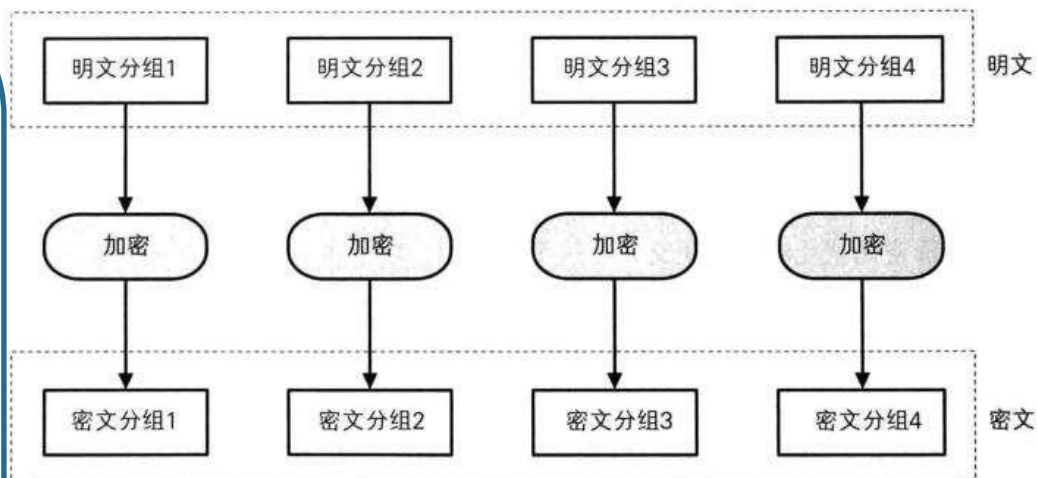
- 电子密码本模式（ECB）。
- 密码块链接模式（CBC）。
- 密文反馈模式（CFB）。
- 输出反馈模式（OFB）。
- 计数器模式（CTR）。



分组密码运行模式

- 电子密码本模式（ECB）
- 各个消息分组独立加密。
- **优点：**加密方与解密方之间的**分组同步不是必须的**，即由于传输问题导致的接收方没有收到所有的加密分组时，不影响已经接收到的分组；各分组之间没有关联，ECB的操作可以**并行化**；
- **缺点：**同样明文块会被加密成相同密文块，因此**不能隐藏数据模式**，不能提供严格的数据保密性，**不推荐用于密码协议中**。

ECB模式的加密



ECB模式的解密

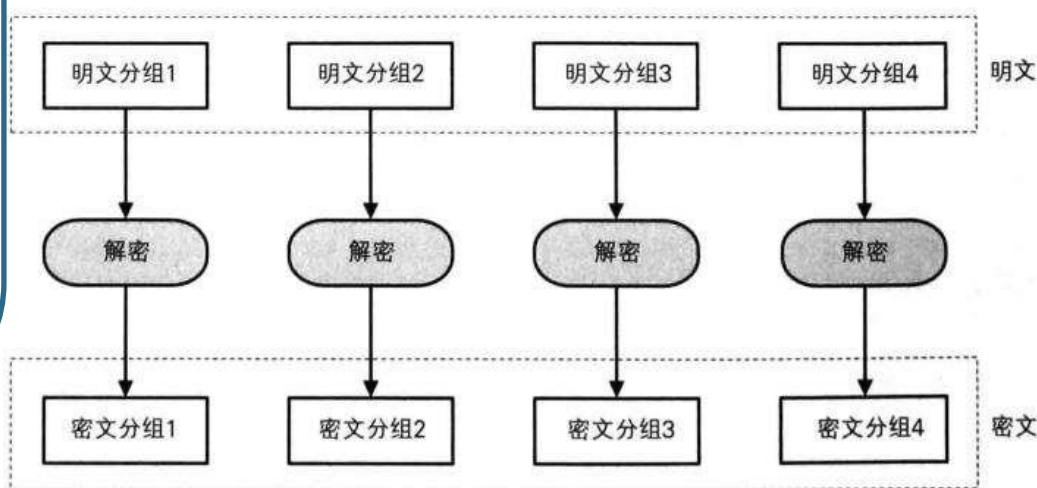


图 4-2 ECB 模式（电子密码本模式）

分组密码运行模式

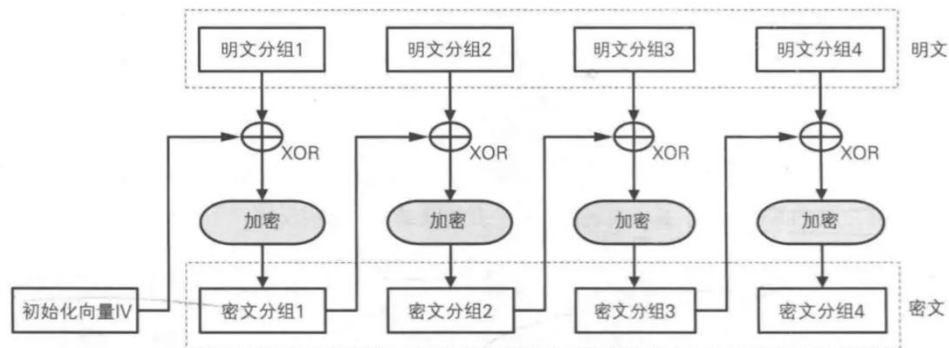
■ 密码块链接模式（CBC）

■ 每个明文消息块先与前一密文块进行异或后，再进行加密。

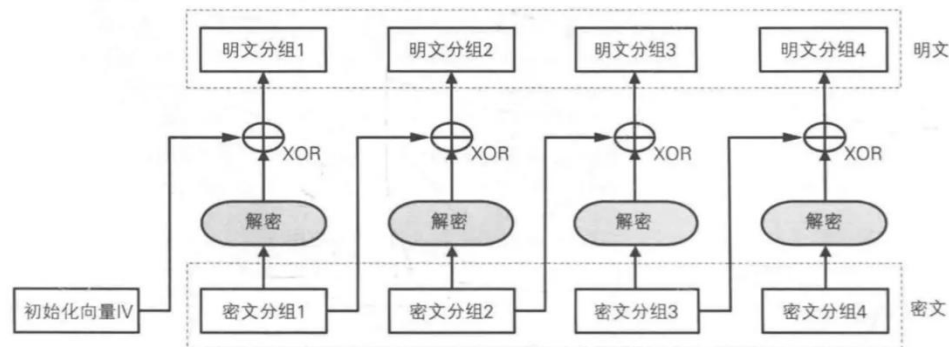
■ **优点：**每次加密都需要前一密文块参与，明文中微小改变会使后续所有密文改变；无法从密文看出明文的规律，**安全性提高，CBC是最为常用的工作模式**；改变某密文块，只会影响当前和下一个解密出的明文，不会影响后续的解密过程。

■ **缺点：**由于每次加密都需要前一个密文参与，所以**无法并行加密**，只能串行加密，**加密速度受到限制**；解密是可以并行进行的。

CBC模式的加密



CBC模式的解密



分组密码运行模式

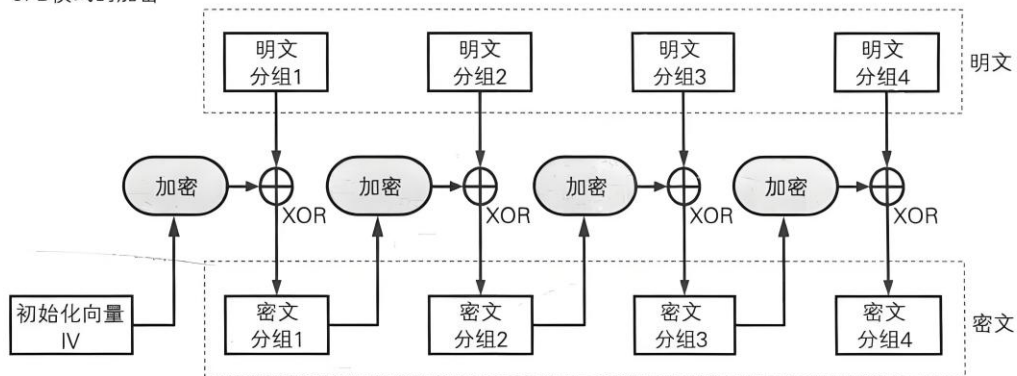
■ 密文反馈模式 (CFB)

■ 加密上一分组密文，然后与当前分组的明文进行“异或”得到当前分组的密文。

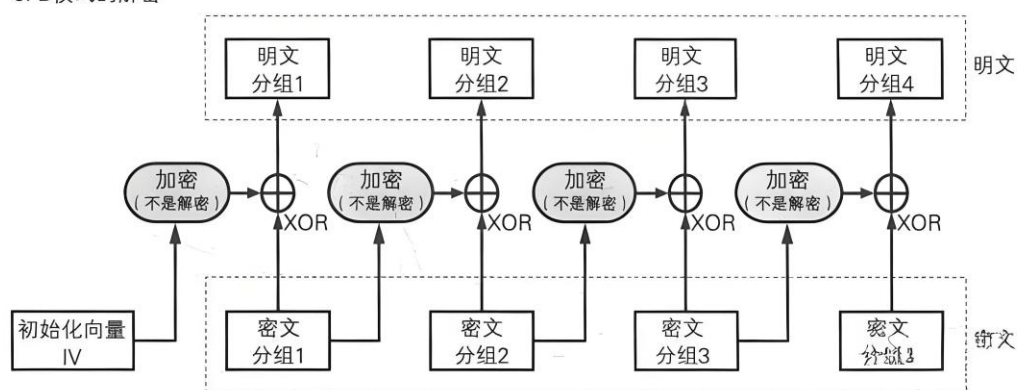
■ **优点：**CFB模式的工作模式与一次性密码本的工作模式非常类似，**安全性较高**；

■ **缺点：**密文输出相当于一次性密码本的随机比特序列，但是由于密码算法的输出是计算得到的，并不是真正的随机数，因此CFB模式**并不能像一次性密码本那样具有理论上的不可破译性**；加密无法并行，解密可以并行。

CFB模式的加密



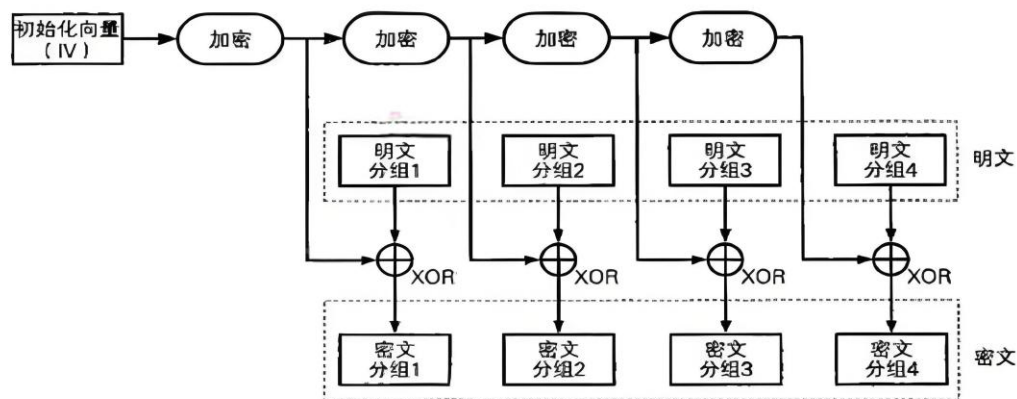
CFB模式的解密



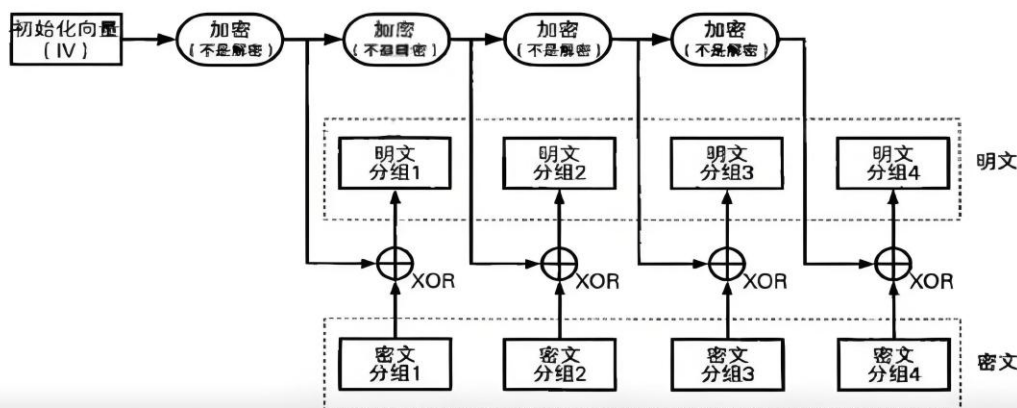
分组密码运行模式

- 输出反馈模式 (OFB)
- 产生密钥流的密文分组，将其与明文分组进行异或，得到密文。
- **优点：**加密前后**不影响**错误校验码（如奇偶校验）的**校验**；
- **缺点：**加解密无法并行。

OFB模式的加密



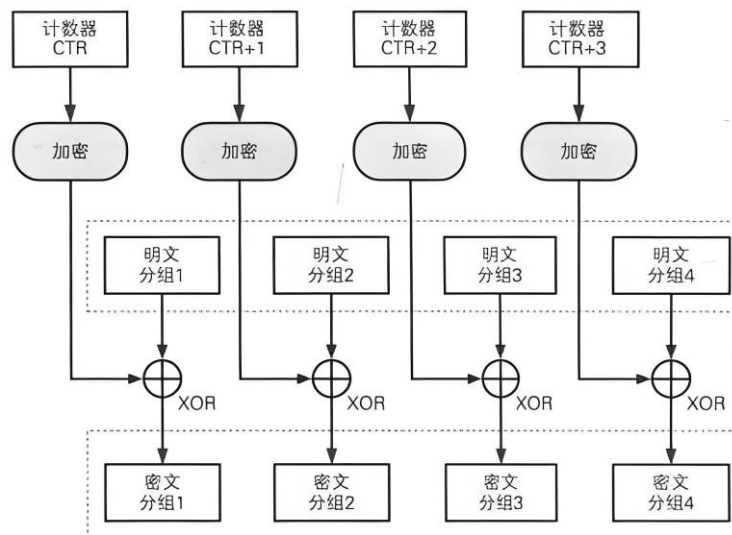
OFB模式的解密



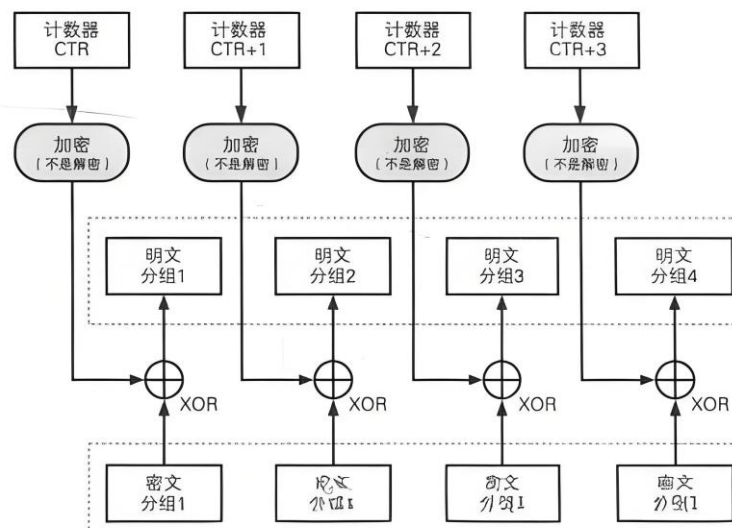
分组密码运行模式

- 计数器模式 (CTR)
- 通过计数器和加密算法产生连续的密钥流，与各个分组明文异或，产生密文。
- 优点：加密和解密使用相同的结构；可以任意顺序对分组进行加密或者解密；加密解密可以并行计算。

CTR模式的加密

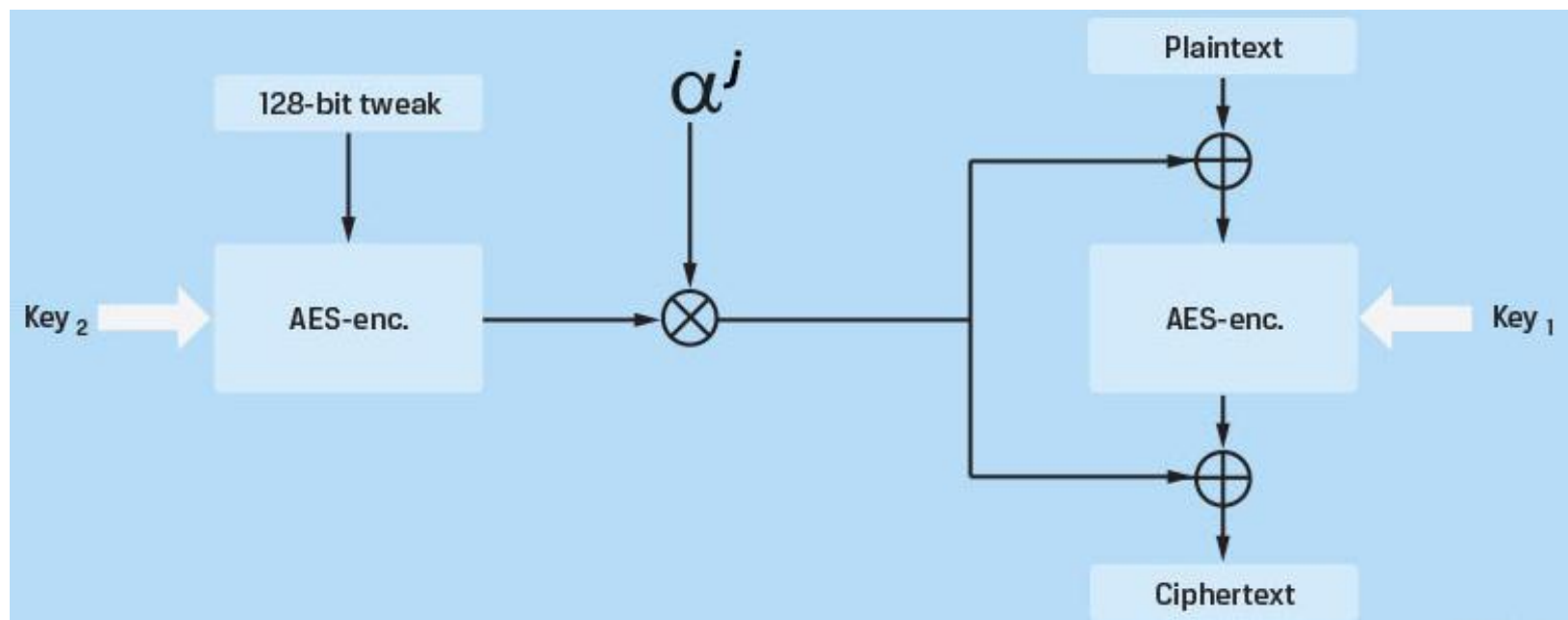


CTR模式的解密



分组密码运行模式

■ 可调整密码本模式 (XTS)



分组密码运行模式

■ 不同分组密码运行模式对比

模式	KEY个数	IV	加密对象	并行处理
ECB	1	无	明文分组	支持（加密、解密）
CBC	1	有	明文分组与上一个密文分组（或初始化向量IV）“异或”后的值	仅解密支持
CFB	1	有	密文分组（或初始化向量IV）	仅解密支持
OFB	1	有	IV值，IV加密的输出作为下一个加密的输入，以此类推	不支持
CTR	1	无	计数器CTR值	支持（加密、解密）
XTS	2	无	明文及tweak值	支持（加密、解密）

分组密码运行模式

不同分组密码运行模式对比

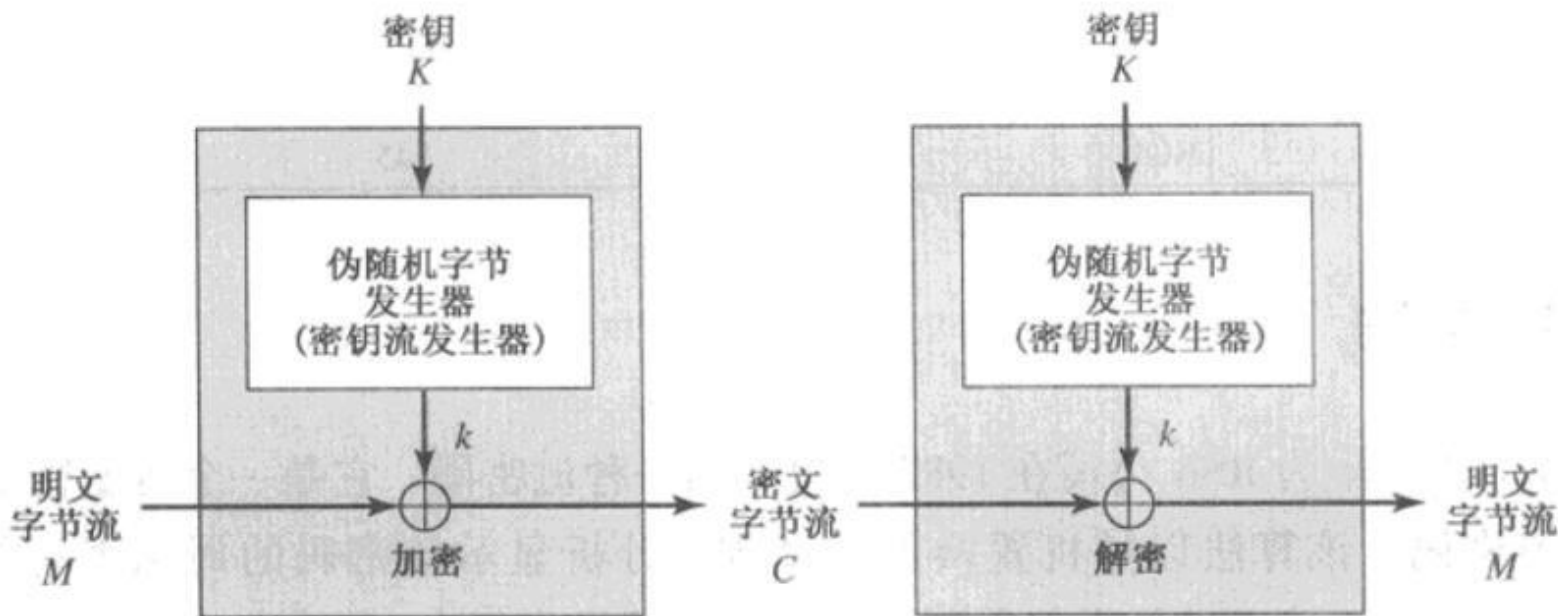
模式	优点	缺点
ECB	1) 简单 2) 快速 3) 支持并行计算（加密、解密）	1) 明文中的重复排列会反映在密文中 2) 通过删除、替换密文分组可以对明文进行操作 3) 不能抵御重放攻击
CBC	1) 明文的重复排列不会反映在密文中 2) 支持并行计算（仅解密） 3) 能够解密任意密文分组	1) 加密不支持并行计算 2) 对包含某些错误的比特的密文进行解密时，第一个分组的全部比特以及后一个分组的相应比特会出错
CFB	1) 不需要填充（padding） 2) 支持并行计算（仅解密） 3) 能够解密任意密文分组	1) 加密不支持并行计算 2) 对包含某些错误的比特的密文进行解密时，第一个分组的全部比特以及后一个分组的相应比特会出错 3) 不能抵御重放攻击
OFB	1) 不需要填充（padding） 2) 可事先进行加密、解密的准备 3) 解密、解密使用相同的结构 4) 对包含某些错误比特的密文进行解密时，只有明文中的相应bit会出错 5) 支持并行计算（加密、解密）	1) 不支持并行计算 2) 主动攻击者反转密文分组中的某些比特时，明文分组中相应的比特也会被翻转
CTR	1) 不需要填充（padding） 2) 可事先进行加密、解密的准备 3) 解密、解密使用相同的结构 4) 对包含某些错误比特的密文进行解密时，只有明文中的相应bit会出错 5) 支持并行计算（加密、解密）	1) 主动攻击者反转密文分组中的某些比特时，明文分组中相应的比特也会被翻转
XTS	1) 支持并行计算（加密、解密） 2) 可事先做好加解密的准备	1) 对包含某些错误的比特的密文进行解密时，第一个分组的全部比特以及后一个分组的相应比特会出错 2) 不能抵御重放攻击

现代流密码算法

- 常用的流密码算法RC4:
- RC4属于**对称密码算法**中的**流密码**加密算法。
- 密钥长度可变，面向**字节**操作。
- 它以一个足够大的S表为基础，对表进行**非线性**变换，产生**流密钥**。

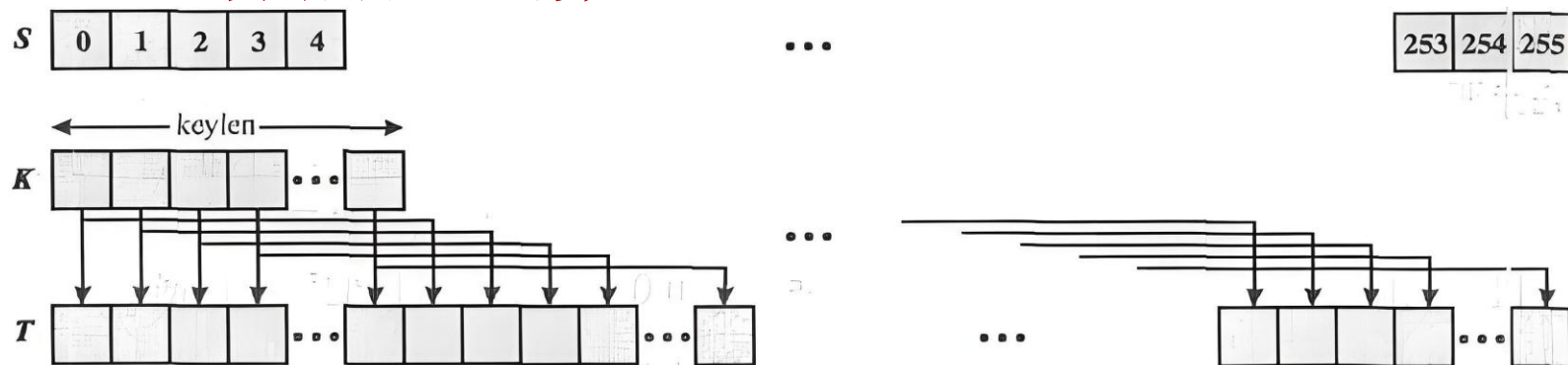
$$\text{密文} = \text{明文} \oplus \text{密钥}$$

$$\text{明文} = \text{密文} \oplus \text{密钥}$$

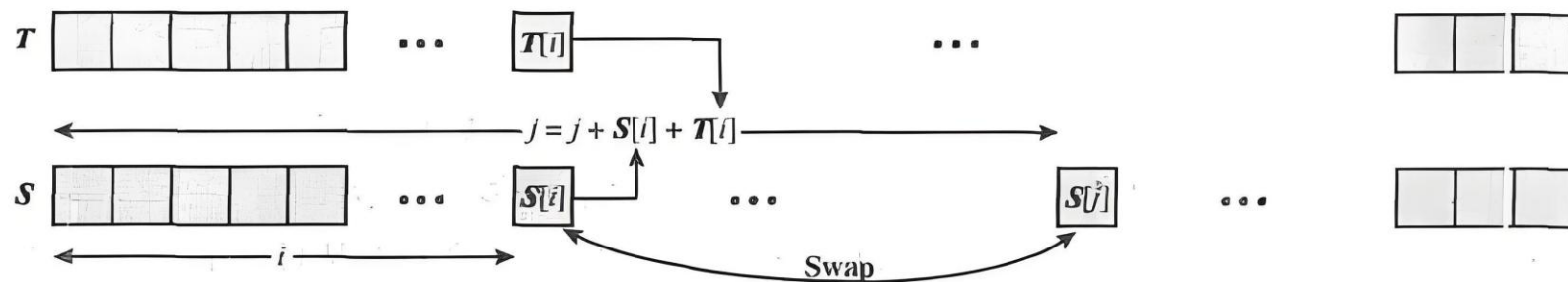


现代流密码算法

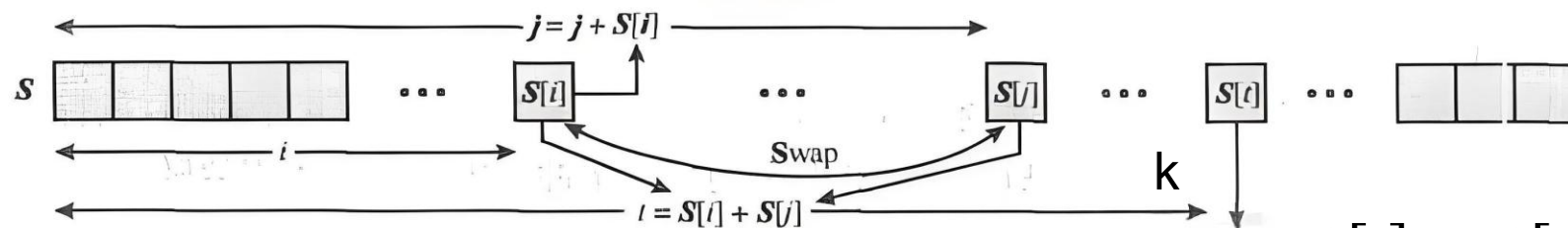
■ 常用的流密码算法RC4: ▶



(a) S 和 T 的初始状态



(b) S 的初始置换

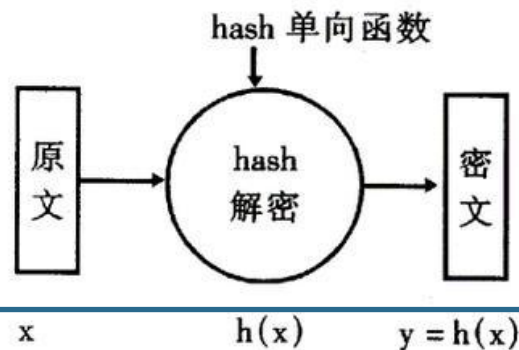


(c) 密钥流的生成

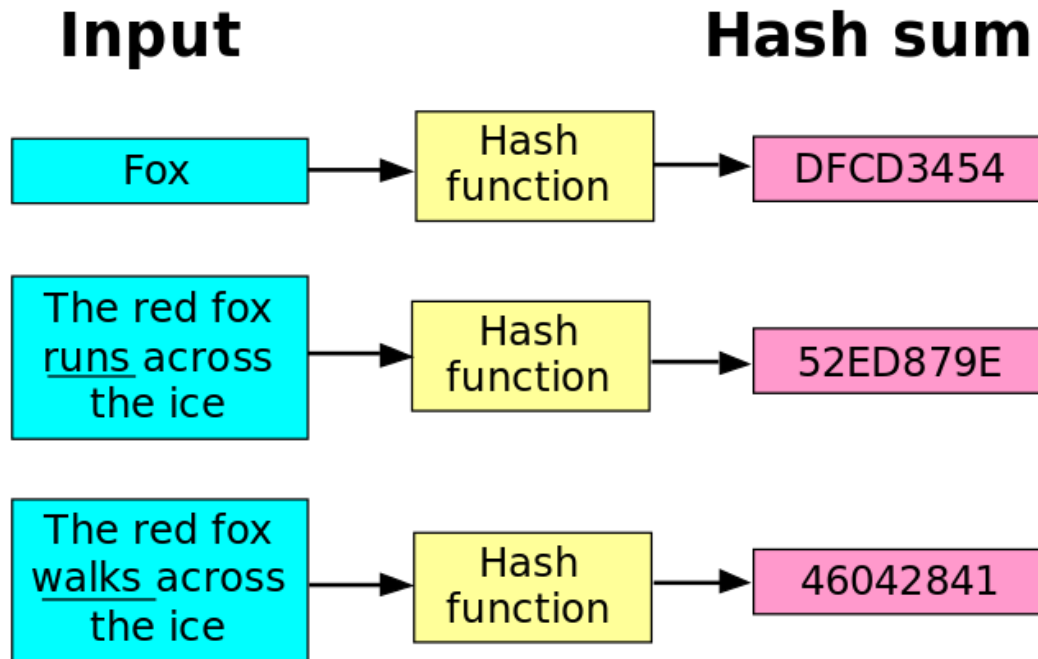
$$C[i] = S[t] \oplus P[i]$$

现代散列算法

- **散列（Hash）算法**在密码学算法中处于基础地位。
- Hash算法**将任意长度的二进制消息转化成固定长度的散列值**。
- Hash算法是一个**不可逆的单向函数**。不同的输入可能会得到相同的输出，而不可能从散列值来唯一的确定输入值。
- 散列函数广泛应用在**密码检验、身份认证、消息认证以及数字签名**上，因此散列函数往往是被应用的最广泛的密码算法。
- 据统计Windows操作系统就需要用到散列算法700多次。
- 常见的散列算法包括MD4、MD5、SHA-1、SHA-2 以及最新的美国国家标准与技术局发布的SHA-3。

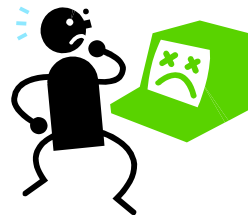


现代散列算法



MD4和MD5

- *MD4 (Message Digest 4)，是麻省理工学院的教授Ronald Rivest 的研究小组在1990年设计的散列算法，因其是他们设计的一系列散列算法中的第4个算法，所以称为MD4。MD4算法是基于 32 位操作数的比特位操作来实现的，其输出散列值长度为128位。该算法设计后被成功破译。
- MD5是 Rivest 于1991年对MD4的改进版本。与MD4算法一样，MD5 算法将输入的信息进行分组，每组仍以512 位（64个 字节），顺序处理完所有分组后输出128 位散列值。在每一组消息的处理中，都要进行4 轮、每轮16 步、总计64 步的处理。其中，每步计算中含一次左循环移位，每一步结束时将计算结果进行一次右循环移位。



MD5

- **MD5算法**的核心是四个非线性函数F、G、H、I和一个128位的缓冲区。具体流程如下：
 - **1. 初始化缓冲区**
 - MD5算法首先需要初始化一个128位的缓冲区，用于存储中间状态和最终结果。缓冲区中包括四个32位的寄存器A、B、C、D，初始值为固定的常数。
 - **2. 填充消息**
 - MD5算法对消息进行填充，以确保消息的长度为512位的倍数。填充规则如下：
 - a. 在消息末尾添加一个1
 - b. 在消息末尾添加若干个0，直到消息长度模512余数为448
 - c. 在消息末尾添加64位的长度，表示原始消息的位数。

- 3. 处理消息分组
- MD5算法将消息分成若干个512位的分组，每个分组包含16个32位的字。对于每个分组，MD5算法按照如下方式进行处理：
 - a. 将缓冲区中的寄存器复制到临时变量中
 - b. 分别执行四轮操作，每轮操作包括16次迭代
 - c. 将每轮操作的结果加到缓冲区的对应寄存器中
 - d. 清空临时变量
- 四轮操作分别使用非线性函数F、G、H、I，以及与轮数相关的常数T，以改变每个字的值。具体操作如下：

MD5

- 对于每轮操作，使用不同的逻辑函数（输入为3个32位字，输出1个32位字）：

轮次	函数	计算公式
第1轮	F	$F(X,Y,Z)=(X\wedge Y)\vee(\neg X\wedge Z)$
第2轮	G	$G(X,Y,Z)=(X\wedge Z)\vee(Y\wedge\neg Z)$
第3轮	H	$H(X,Y,Z)=X\oplus Y\oplus Z$
第4轮	I	$I(X,Y,Z)=Y\oplus(X\vee\neg Z)$

- 常数表T

$$T[i] = \lfloor 2^{32} \times |\sin(i)| \rfloor, i = 1, 2, \dots, 64$$

MD5

- 每轮16次迭代使用消息分组 $M[0..15]$ 的不同顺序：

轮次	字使用顺序（k值）
第1轮	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
第2轮	1, 6, 11, 0, 5, 10, 15, 4, 9, 14, 3, 8, 13, 2, 7, 12
第3轮	5, 8, 11, 14, 1, 4, 7, 10, 13, 0, 3, 6, 9, 12, 15, 2
第4轮	0, 7, 14, 5, 12, 3, 10, 1, 8, 15, 6, 13, 4, 11, 2, 9

- 循环左移位表s

轮次	移位位数（对应16次迭代）
第1轮	7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22
第2轮	5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20
第3轮	4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23
第4轮	6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21

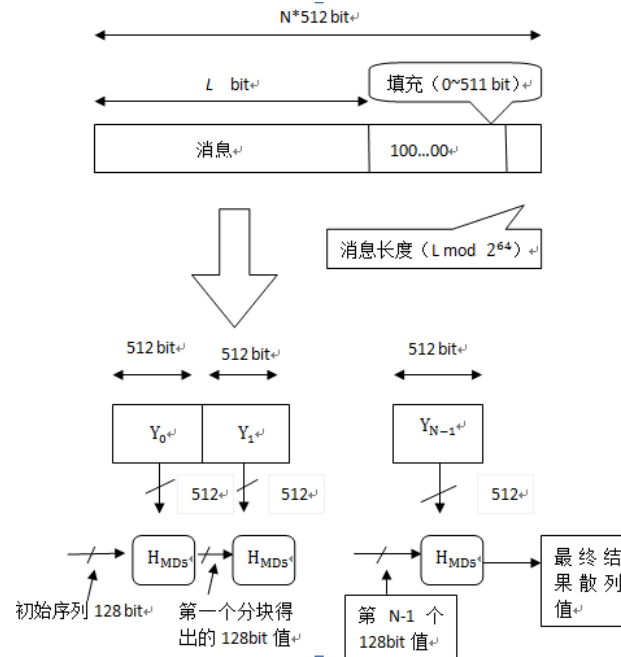
MD5

- 对于第 i 次迭代 ($i=1..64$) :
 - $\text{temp} = D$
 - $D = C$
 - $C = B$
 - $B = B + \text{LEFT_ROTATE}((A + \text{func}(B, C, D) + M[k] + T[i]), s)$
 - $A = \text{temp}$
- 所有加法均为模 2^{32} 加法

MD5

■ 4. 生成消息摘要

- MD5算法对所有分组处理完毕后，将缓冲区中的寄存器拼接成一个128位的二进制数（小端序），即为消息摘要。MD5算法的输出结果是一个固定长度的128位哈希值，用于校验数据的完整性和真实性。



MD4和MD5

- MD5 比 MD4 复杂，并且速度较之要慢一些，但更安全，在抗分析和抗差分方面表现更好。MD5-Hash 文件的数字文摘通过 Hash 函数计算得到。不管文件长度如何，它的 Hash 函数计算结果都是一个固定长度的数字。采用安全性高的 Hash 算法，两个不同的文件几乎不可能得到相同的 Hash 结果。因此，一旦文件被修改，就可被检测出来。
- MD5（散列）算法威胁（安全性要求）
 - 原象攻击：给定一个哈希值 h ，找到任意消息 m 使得 $H(m)=h$
 - 第二原象攻击：给定消息 m_1 及其哈希值 $h=H(m_1)$ ，找到另一个不同的消息 m_2 不等于 m_1 使得 $H(m_2)=h$
 - 碰撞攻击：找到任意两个不同的消息 m_1 和 m_2 ，使得 $H(m_1)=H(m_2)$ （相同前缀碰撞、选择前缀碰撞）



安全散列算法SHA, SHA-1, SHA-2 和 SHA-3

- *1993年，美国国家安全局（NSA）和美国国家标准技术局（NIST）共同提出了安全散列算法SHA， 并作为联邦信息处理标准（FIPS PUB 180）公布
- 1995年又发布了一个修订版FIPS PUB 180-1，通常称之为SHA-1。SHA-1是基于MD4算法的，并且它的设计在很大程度上是模仿MD4的。SHA-1 输出160比特的散列值，已得到广泛的应用。
- 随着MD5散列算法的破解，SHA-1的安全性也受到了质疑。NIST建议之后的商业软件产品由SHA-1转移到SHA-2散列算法上。SHA-2 是一类可变长度的散列算法，其包含SHA-256， SHA-384和SHA-512，分别输出256、384和512比特的散列值。

安全散列算法SHA, SHA-1, SHA-2 和 SHA-3

- *由于SHA-1和SHA-2的设计思路都与MD5类似，所以他们可能受到同一种攻击的威胁。所以NIST于2007年发起了一轮新的安全散列算法标准的竞赛，共有64个算法参加了竞赛。
- 最终在2012年由来自意法半导体公司的Joan Daemen（此人也是AES算法的设计者）领导的小组设计的Keccak算法胜出，成为了新一代的散列算法标准SHA-3。



安全散列算法SHA, SHA-1, SHA-2 和 SHA-3

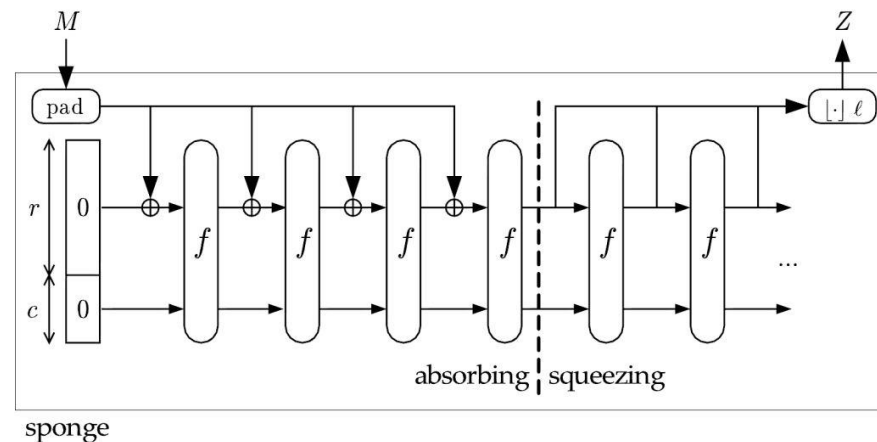
■ Keccak海绵函数

● 吸收阶段 (Absorbing) :

- 将输入消息分块与状态矩阵 (5x5的64位矩阵, 1600位容量) 进行异或
- 应用置换函数 f (Keccak- f)

● 挤压阶段 (Squeezing) :

- 从状态矩阵提取哈希结果
- 多次应用置换函数直到输出足够长度





安全散列算法SHA, SHA-1, SHA-2 和 SHA-3

■ SHA-3

算法变种	输出长度	容量c	比特率r	轮数
SHA3-224	224	448	1152	24
SHA3-256	256	512	1088	24
SHA3-384	384	768	832	24
SHA3-512	512	1024	576	24

安全散列算法SHA, SHA-1, SHA-2 和 SHA-3

■ SHA-3

- 消息填充规则
 - 使用pad10*1填充方案
- 置换函数Keccak-f
 - 包含5个核心步骤（每轮执行一次）：
 - θ (Theta)：基于列的非线性扩散
 - ρ (Rho)：位循环移位
 - π (Pi)：行位置置换
 - χ (Chi)：非线性替换
 - ι (Iota)：与轮常数异或

5.3.2 非对称加密体制

- **对称密码体制**的**缺点**是每对通信双方必须共享一个密钥,使得密钥管理复杂。为解决此类问题,1976年,美国斯坦福两位学者Diffie和Hellman提出了著名的非对称密码体制。
- **非对称密码体制**可有效的用在密钥管理、加密和数字签名上非对称加密算法**需要两个密钥**: 公开密钥和私有密钥,并且相互关联。如果用公开密钥对数据进行加密,只有用对应的私有密钥才能解密; 如果用私有密钥对数据进行加密,那么只有用对应的公开密钥才能解密。
- 典型实用的非对称加密算法有RSA、Elgamal和 ECC (椭圆曲线算法)。



5.3.2 非对称加密体制——RSA算法

导入





5.3.2 非对称加密体制——RSA算法

导入





导入



- RSA算法是如何对文件进行加密的？
- 被加密的文件为什么难以解锁呢？



5.3.2 非对称加密体制——RSA算法

RSA算法



理论模型



密码系统构造



执行过程



算法论证



数论基础

1. 质数

一个大于1的自然数，除了1和它本身外，不能被其他自然数整除（除0以外）的数称之为质数（素数）；否则称为合数。

2. 互质数

互质，又称互素。若N个整数的最大公约数是1，则称这N个整数互质。

3. 最大公约数（GCD）

最大公约数（Greatest Common Divisor, GCD）是两个或多个整数共有的最大因数。例如： $\text{GCD}(12, 18) = 6$ 、 $\text{GCD}(15, 25) = 5$ 。



数论基础

3.模运算

模运算的基本思想是将一个整数除以某个模数（modulus），并取余数作为结果。例如：

$a \bmod n$ 表示 a 除以 n 后的余数。

例如： $17 \bmod 5 = 2$ ，因为 $17 \div 5 = 3$ 余 2。

模运算的性质

$$(a+b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$$

$$(a \cdot b) \bmod n = [(a \bmod n) \cdot (b \bmod n)] \bmod n$$



数论基础

4. 同余

给定一个正整数 m ，如果两个整数 a 和 b 满足 $a-b$ 能被 m 整除，即 $(a-b) \bmod m = 0$ ，那么就称整数 a 与 b 对模 m 同余，记作 $a \equiv b \pmod{m}$ ，同时可成立 $a \bmod m = b$ 。

同余性质：

1. 反身性： $a \equiv a \pmod{m}$;
2. 对称性： 若 $a \equiv b \pmod{m}$ ，则 $b \equiv a \pmod{m}$;
3. 传递性： 若 $a \equiv b \pmod{m}$ ， $b \equiv c \pmod{m}$ ，则 $a \equiv c \pmod{m}$;
4. 同余式相加： 若 $a \equiv b \pmod{m}$ ， $c \equiv d \pmod{m}$ ，则 $a \pm c \equiv b \pm d \pmod{m}$;
5. 同余式相乘： 若 $a \equiv b \pmod{m}$ ， $c \equiv d \pmod{m}$ ，则 $ac \equiv bd \pmod{m}$ 。



数论基础

5.模反元素（模逆元）

- 定义：如果两个正整数 e 和 $\varphi(n)$ 互质，那么一定可以找到一个整数 d ，使得 $ed-1$ 被 $\varphi(n)$ 整除，或者说 ed 除以 $\varphi(n)$ 的余数为1，此时， d 叫做 e 的模反元素，即：
 - $ed-1 = k \varphi(n)$
 - $ed \bmod \varphi(n) = 1$
 - $ed \equiv 1 \pmod{\varphi(n)}$
 - 举例：两个互质正整数为3和11，3的模反元素为4
 - 因为 $3*4 - 1 = 11$
- 模反元素不是唯一的， $d \pm k \cdot \varphi(n)$ 都是 e 的模反元素。



数论基础

6. 欧拉函数

- 定义：欧拉函数 $\varphi(n)$ 为小于 n 的正整数中与 n 互质的数的数目。
 - 举例： $\varphi(6)=2$ ，小于6的正整数：1、2、3、4、5
 - 举例： $\varphi(7)=6$ ，小于7的正整数：1、2、3、4、5、6
- 定理：质数 n 的欧拉函数 $\varphi(n)=n-1$ 。
 - 质数 n 与所有小于它的正整数互质
- 定理：如果 n 可以分解为2个互质整数之积，那么 n 的欧拉函数等于这两个因子的欧拉函数之积。
 - 结合上述两个定理， $\varphi(n)=\varphi(p*q)=\varphi(p)*\varphi(q)=(p-1)*(q-1)$



数论基础

欧拉定理：

设 n 是大于1的整数，若整数 a 满足 $(a, n) = 1$ ，则

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

其中 $\varphi(n)$ 为欧拉函数，即 $1 \sim n$ 中与 n 互素的整数的个数。

费马小定理：

设 p 是一个素数，则对任意整数 a ，我们有

$$a^p \equiv a \pmod{p}$$



数论基础

中国剩余定理Chinese Remainder Theorem有很多种表述，但最根本、最简化的原理为：

$$\left. \begin{array}{l} a \equiv b \pmod{m} \\ a \equiv b \pmod{n} \end{array} \right\} a \equiv b \pmod{[m, n]}$$

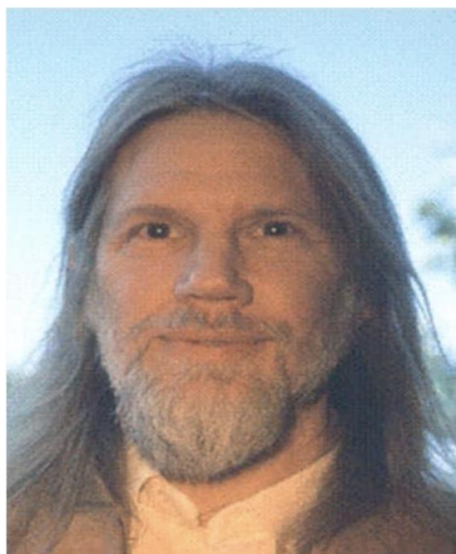
这也是同余性质（公式）9，在此基础上，可以循序渐进的推出三个原理：

- ①**基础版**：若两个模数互质，则模乘积范围内，任意不同数对两个模的余数组合不同。
- ②**通用版**：若多个模数两两互质，则模总乘积范围内，任意不同数对各个模的余数组合不同。
- ③**最大兼容版**：任意多个模数，则模最小公倍数范围内，任意不同数对各个模的余数组合不同。



理论模型

密码学的新方向（New Directions in Cryptography）



Whitfield Diffie (1944~)



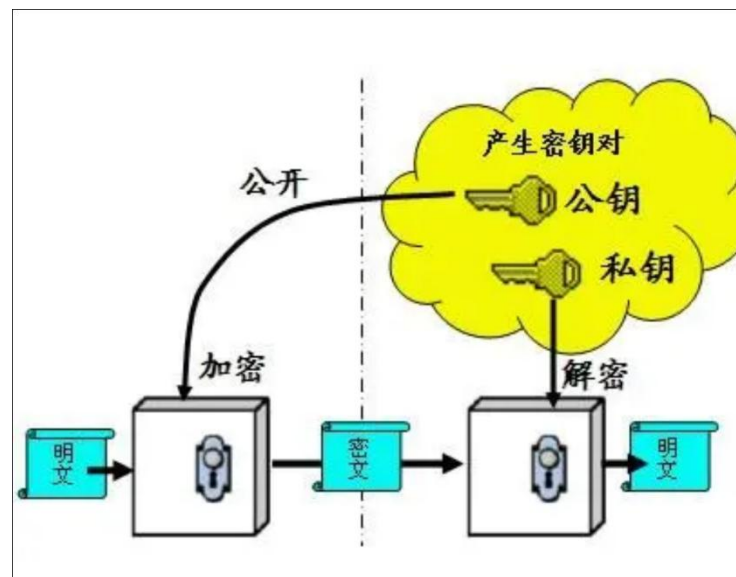
Martin Hellman (1945~)

对称密码体制在密钥分配和数字签名中存在不足



理论模型

- 算法的设计基于数学函数
- 采用两个密钥
 - 公开密钥 (公钥 PK, Public Key)
 - 秘密密钥 (私钥 SK, Private Key)



应用最广泛的非对称加密算法——RSA算法



5.3.2 非对称加密体制——RSA算法

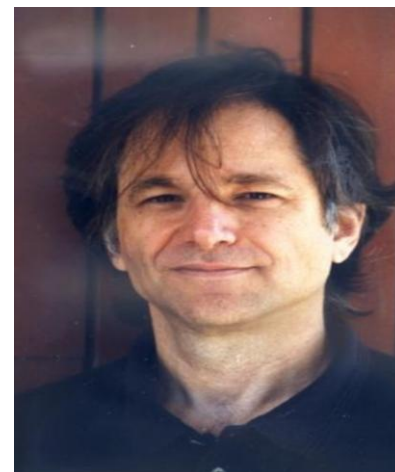
理论模型



Ronald L. Rivest
(1947~)



Adi Shamir
(1952~)



Leonard M. Adleman
(1945~)

2002年他们被授予图灵奖



理论模型

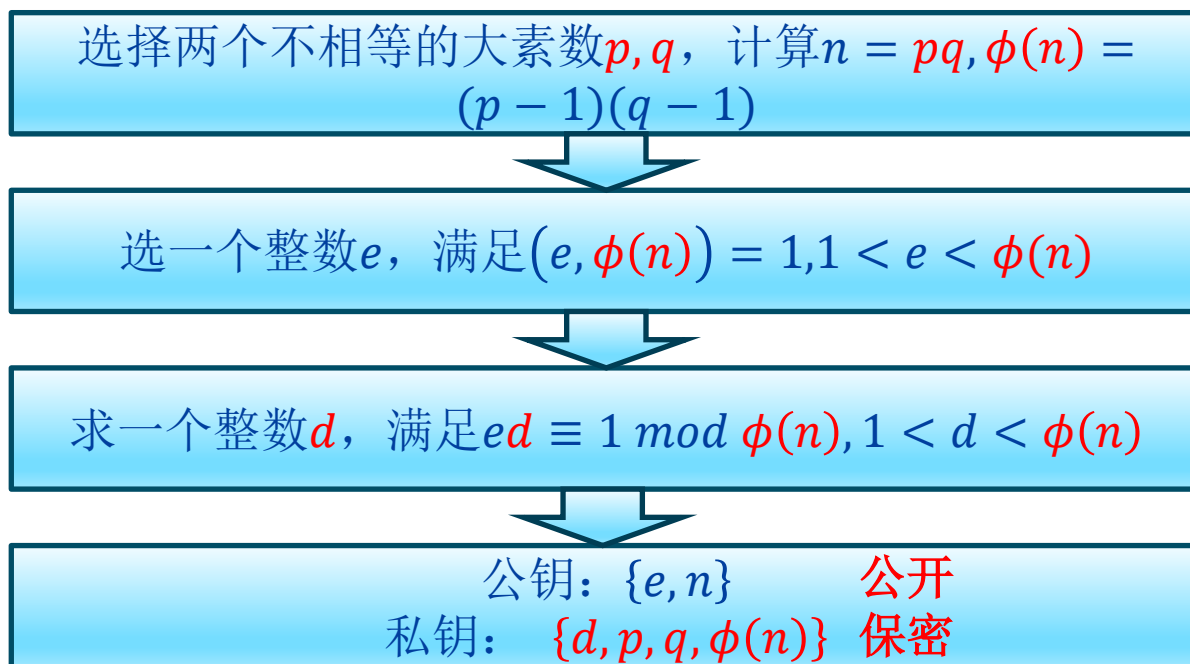
RSA 算法的理论模型：依赖于大整数因子分解的困难性



同学们还能想到单向函数的例子吗?



密码系统构造



广义Euclidean算法

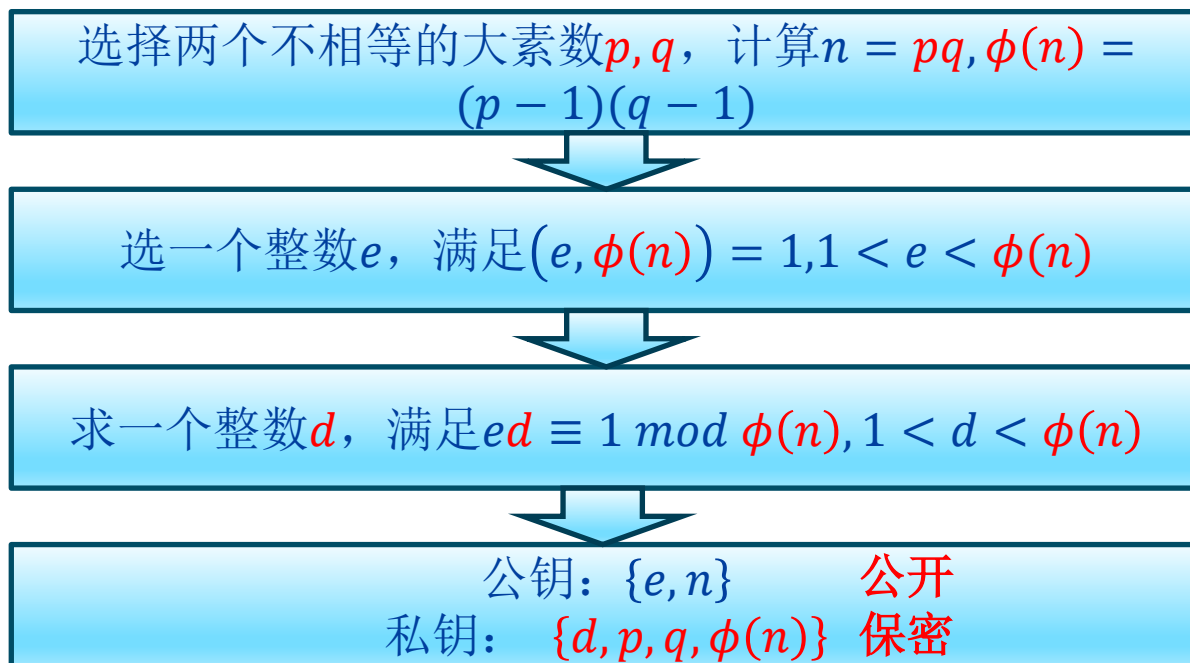


计算n的欧拉函数

- 定义：欧拉函数 $\varphi(n)$ 为小于n的正整数中与n互质的数的数目。
 - 举例： $\varphi(6)=2$ ，小于6的正整数：1、2、3、4、5
 - 举例： $\varphi(7)=6$ ，小于7的正整数：1、2、3、4、5、6
- 定理：质数n的欧拉函数 $\varphi(n)=n-1$ 。
 - 质数n与所有小于它的正整数互质
- 定理：如果n可以分解为2个互质整数之积，那么n的欧拉函数等于这两个因子的欧拉函数之积。
 - 结合上述两个定理， $\varphi(n)=\varphi(p*q)=\varphi(p)*\varphi(q)=(p-1)*(q-1)$



密码系统构造



广义Euclidean算法



计算模反元素d

- 定义：如果两个正整数 e 和 $\varphi(n)$ 互质，那么一定可以找到一个整数 d ，使得 $ed-1$ 被 $\varphi(n)$ 整除，或者说 ed 除以 $\varphi(n)$ 的余数为1，此时， d 叫做 e 的模反元素，即：
 - $ed-1 = k \varphi(n)$
 - $ed \bmod \varphi(n) = 1$
 - $ed \equiv 1 \pmod{\varphi(n)}$
 - 举例：两个互质正整数为3和11，3的模反元素为4
 - 因为 $3*4 - 1 = 11$
- 模反元素不是唯一的， $d \pm k \cdot \varphi(n)$ 都是 e 的模反元素。



欧几里得算法

- 欧几里德算法又称辗转相除法，主要是用于计算两个整数 a, b 的最大公约数。
- 简单点说一下算法原理：两个整数的最大公约数等于其中小的数跟大除以小余数的最大公约数： $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 比如求 10 跟 24 的最大公约数 $a = \gcd(10, 24)$:
 1. 求 10 和 24 的最大公约数等于求 10 跟 4 的最大公约数：
$$a = \gcd(10, 24) = \gcd(10, 4)$$
 2. 求 10 跟 4 的最大公约数等于求 4 跟 2 的最大公约数，为 2：
$$a = \gcd(10, 24) = \gcd(10, 4) = \gcd(4, 2) = 2$$



广义（扩展）欧几里得算法

- 算法原理：若 a 和 b 为正整数，则存在整数 x, y 使得

$$\gcd(a,b)=ax+by$$

- 通俗点说就是 $\gcd(a,b)$ 可以表示为 a,b 的整数线性组合。
- 举个简单的例子：
- $\gcd(10, 24) = 2$
- $2 = 10*(-7) + 24*3$



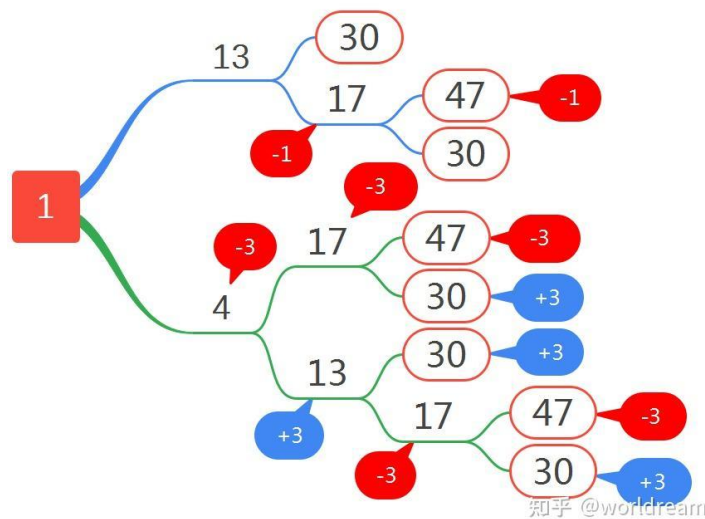
广义（扩展）欧几里得算法求解模的逆元

- 求解步骤：
 - 检查互质性：首先需要确认 a 和 m 是否互质（即 $\gcd(a, m) = 1$ ）。如果互质，则存在逆元；否则不存在。
 - 应用扩展欧几里得算法：扩展欧几里得算法可以找到整数 x 和 y ，使得 $a \cdot x + m \cdot y = \gcd(a, m)$ 。当 $\gcd(a, m) = 1$ 时，解 x 就是 a 在模 m 下的逆元。
 - 归一化结果：由于扩展欧几里得算法可能返回负数，需要将结果调整为模 m 的正整数等价类。



● 示例

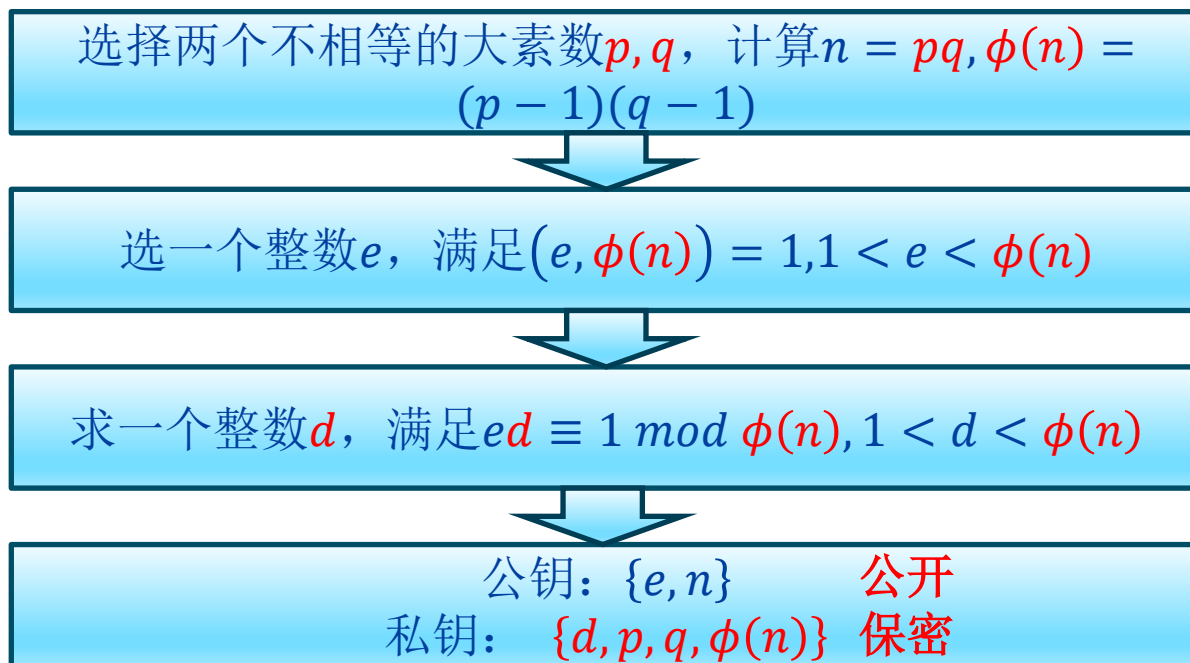
- 假设需要求解 30 在模 47 下的逆元:
- 确认互质性: $\gcd(30,47)=1$, 说明存在逆元。



- 最终得到 $30 \cdot 11 + 47 \cdot (-7) = 1$ ，因此 11 是 30 在模 47 下的逆元。
- 即： $11 * 30 = 1 \pmod{47}$



密码系统构造



广义Euclidean算法



执行过程

加密 (E) :

计算 $C \equiv M^e \bmod n$

解密 (D) :

计算 $M \equiv C^d \bmod n$



示例:

- 选择两个素数 p 和 q :

$$p=11$$

$$q=13$$

- 计算模数 n :

$$n=p \times q=11 \times 13=143$$

- 计算欧拉函数 $\phi(n)$:

$$\phi(n)=(p-1)(q-1)=10 \times 12=120$$



示例:

- 选择公钥指数 e :

选择 $e=7$ ，因为 $1 < 7 < 120$ 且 7 与 120 互质。

- 计算私钥指数 d :

需要找到 d 使得 $d \times e \equiv 1 \pmod{\phi(n)}$ 。

使用扩展欧几里得算法，可以找到 $d=103$ ，因为 $7 * 103 \pmod{120} = 1$ 。

公钥是 (7,143)，私钥是 (103,143)。



示例：

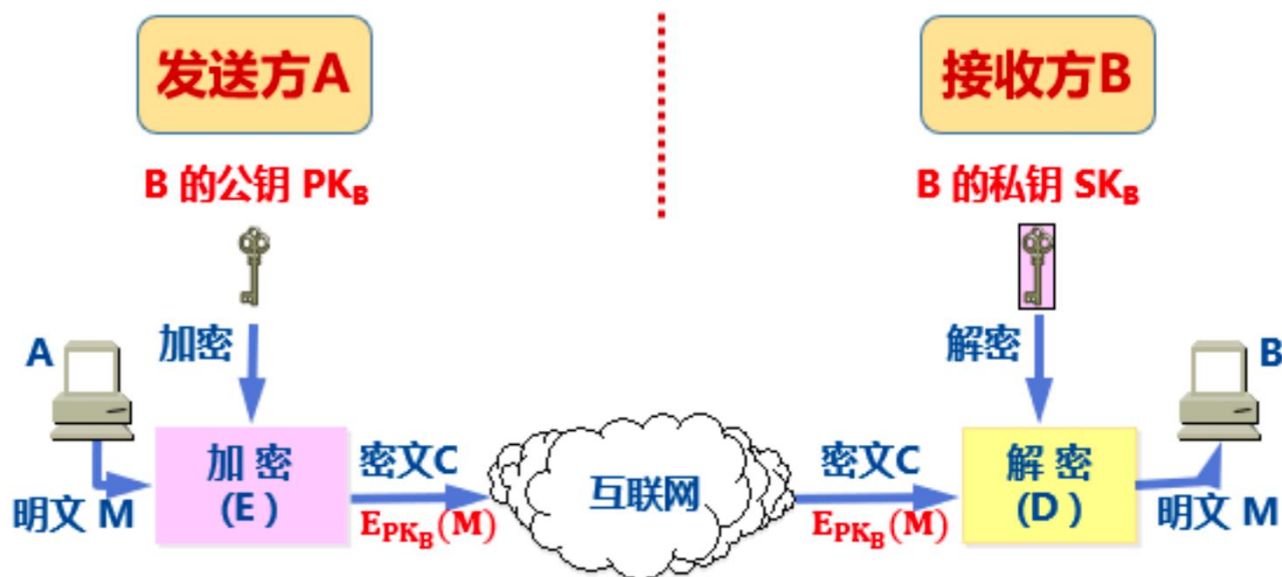
- 假设明文 $M=9$ 。
- 加密： $C = M^e \bmod n = 9^7 \bmod 143 \quad C = 48$
CSDN @licy_
- 解密： $M = C^d \bmod n = 48^{103} \bmod 143 \quad M = 9$
CSDN @licy_
- 通过以上步骤，我们成功地完成了RSA算法的加密和解密过程。



5.3.2 非对称加密体制——RSA算法

执行过程——数据保密性

实现多个用户加密的数据只能由合法的用户解密

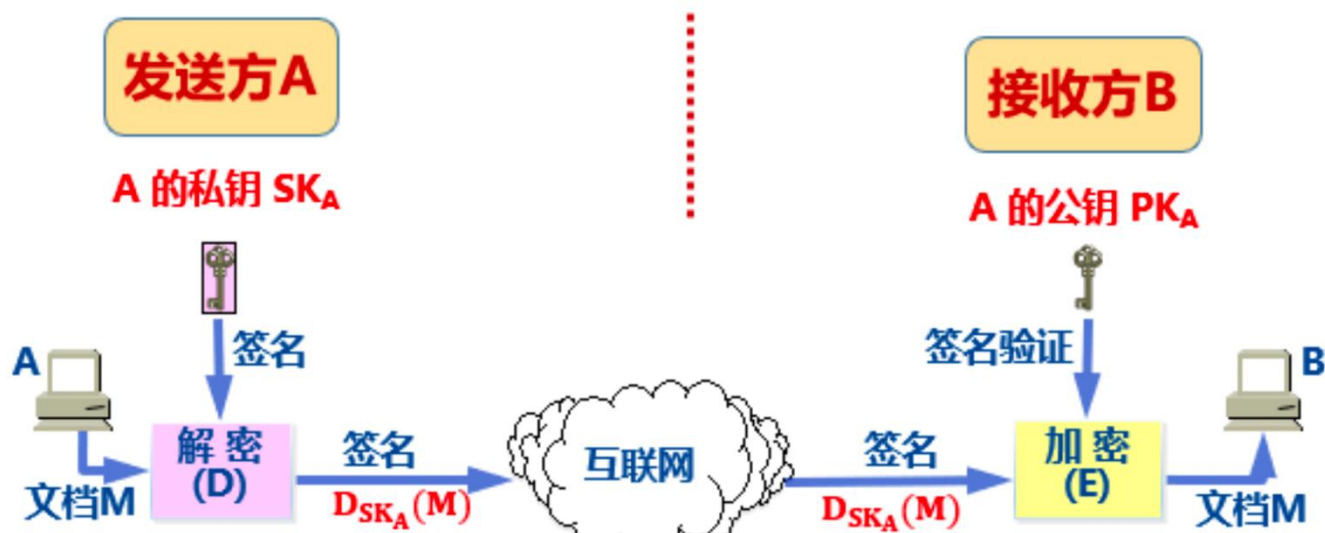




5.3.2 非对称加密体制——RSA算法

执行过程——数据真实性

实现一个用户签名的消息多个用户可以验证



算法论证1——可逆性

- 证明： 明文： M 加密： $M^e \bmod n = C$
 密文： C 解密： $C^d \bmod n = M$

$$\begin{aligned} & C^d \bmod n && \text{➤证明密文C解密结果为M} \\ &= (M^e \bmod n)^d \bmod n && \text{➤带入加密过程} \\ &= M^{ed} \bmod n && \text{➤} ab \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n \\ &= M^{k\varphi(n)+1} \bmod n && \text{➤} ed \bmod \varphi(n) = 1 \\ &= M^{k\varphi(n)} \cdot M \bmod n && \text{➤幂次拆开} \\ &= ((M^{k\varphi(n)} \bmod n) \cdot (M \bmod n)) \bmod n && \text{➤取模运算乘法公式} \\ &= ((M^{k\varphi(n)} \bmod n) \cdot M) \bmod n && \text{➤} n \text{远远大于} M \\ &= 1? \end{aligned}$$

算法论证1——可逆性

明文： M 加密： $M^e \bmod n = C$

密文： C 解密： $C^d \bmod n = M$

- 欧拉定理：如果 a 和 n 互质，那么 $a^{\varphi(n)} \equiv 1 \bmod n$ 。

➤ 即： $a^{\varphi(n)} \bmod n = 1$

$M^{k\varphi(n)} \bmod n$ ➤ 证明该式等于1

$= (M^k)^{\varphi(n)} \bmod n = 1$ ➤ 利用上述定理

M^k, n 互质？ $n = pq$ 且 p, q 互质 $\Rightarrow$ n 的因子只有 $1, n, p, q$

$M^k \ll p, q$ $\Rightarrow$ M^k, n 互质

算法论证2——可逆性

● 证明： 明文： M 加密： $M^e \bmod n = C$
 密文： C 解密： $C^d \bmod n = M$

$$\begin{aligned} C^d \bmod n & \quad \text{➤证明密文C解密结果为M} \\ = (M^e \bmod n)^d \bmod n & \quad \text{➤带入加密过程} \\ = M^{ed} \bmod n & \quad \text{➤} ab \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n \end{aligned}$$

- 假设 $ed \equiv 1 \pmod{(p-1)(q-1)}$ ，即存在整数 k 使得 $ed = 1 + k(p-1)(q-1)$ 。
- 当 $m \not\equiv 0 \pmod p$ 时，根据费马小定理， $m^{p-1} \equiv 1 \pmod p$ ，因此 $m^{ed} = m^{1+k(p-1)(q-1)} = m \cdot (m^{p-1})^{k(q-1)} \equiv m \cdot 1^{k(q-1)} \equiv m \pmod p$ 。
- 同理，当 $m \not\equiv 0 \pmod q$ 时， $m^{q-1} \equiv 1 \pmod q$ ，因此 $m^{ed} \equiv m \pmod q$ 。
- 通过中国剩余定理，可以得出 $m^{ed} \equiv m \pmod n$ ，从而证明了解密过程的正确性。



算法论证——可逆性

要证明: $M = D(E(M))$

即 $M = C^d = (M^e)^d = M^{ed} \bmod n$

$\because ed = 1 \bmod \phi(n)$, 即 $ed = t\phi(n) + 1$, t 为整数

$\therefore M^{ed} = M^{t\phi(n)+1} \bmod n$

要证明 $M^{ed} = M \bmod n$, 只需证明

$$M^{t\phi(n)+1} = M \bmod n$$

(根据欧拉定理可证明)

保证了RSA算法的数据安全性



算法论证——可交换性

$$D(E(M)) = (M^e)^d = M^{ed} = (M^d)^e = E(D(M)) \bmod n$$

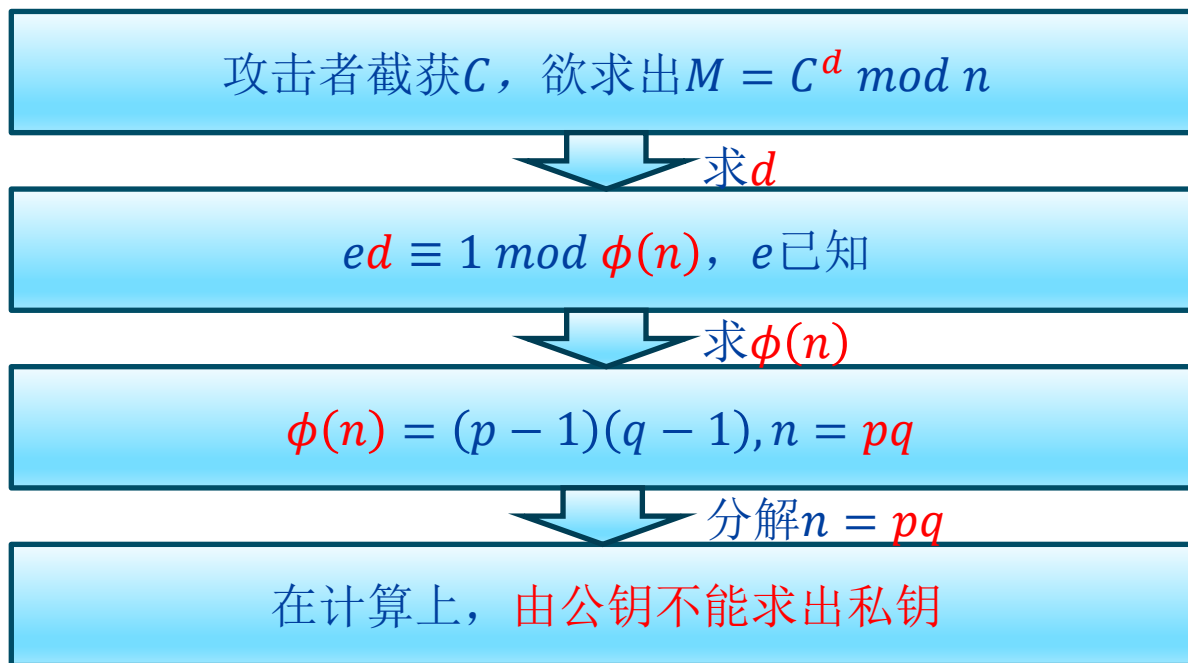
因为 M^{ed} 的指数 ed 是满足乘法交换律性的

因此， $M^{ed} = M^{de}$ 。

保证了RSA算法的数据真实性



算法论证——由公钥不能计算私钥



大整数因子分解的困难性



算法论证——参数选择

参数选择的是否合适直接影响到算法的安全性

p 和 q 到底取多大?

一般应用: p 和 q 应为**512bit**, n 为**1024bit**

重要应用: p 和 q 应为**1024bit**, n 为**2048bit**

RSA算法

- *RSA算法的优点是应用更加广泛，而其缺点是加密速度慢。
- 如果 RSA 和 AES 结合使用，则正好弥补 RSA 的缺点。即 AES 用于明文加密，RSA 用于 AES 的密钥加密。由于AES加密速度快，适合加密较长的消息；而 RSA 可解决 AES 密钥传输问题。
- 非对称密码算法大多是基于数学困难问题，如大整数因子分解（RSA）、离散对数难题（DSA）、椭圆曲线离散对数难题（ECC）等等，运行速度较慢。非对称密码算法与同等安全强度的对称密码算法相比，一般要慢三个数量级。因此非对称密码算法一般用来加密短数据或者用作数字签名，而不是直接用在数据加密上。

ECC算法

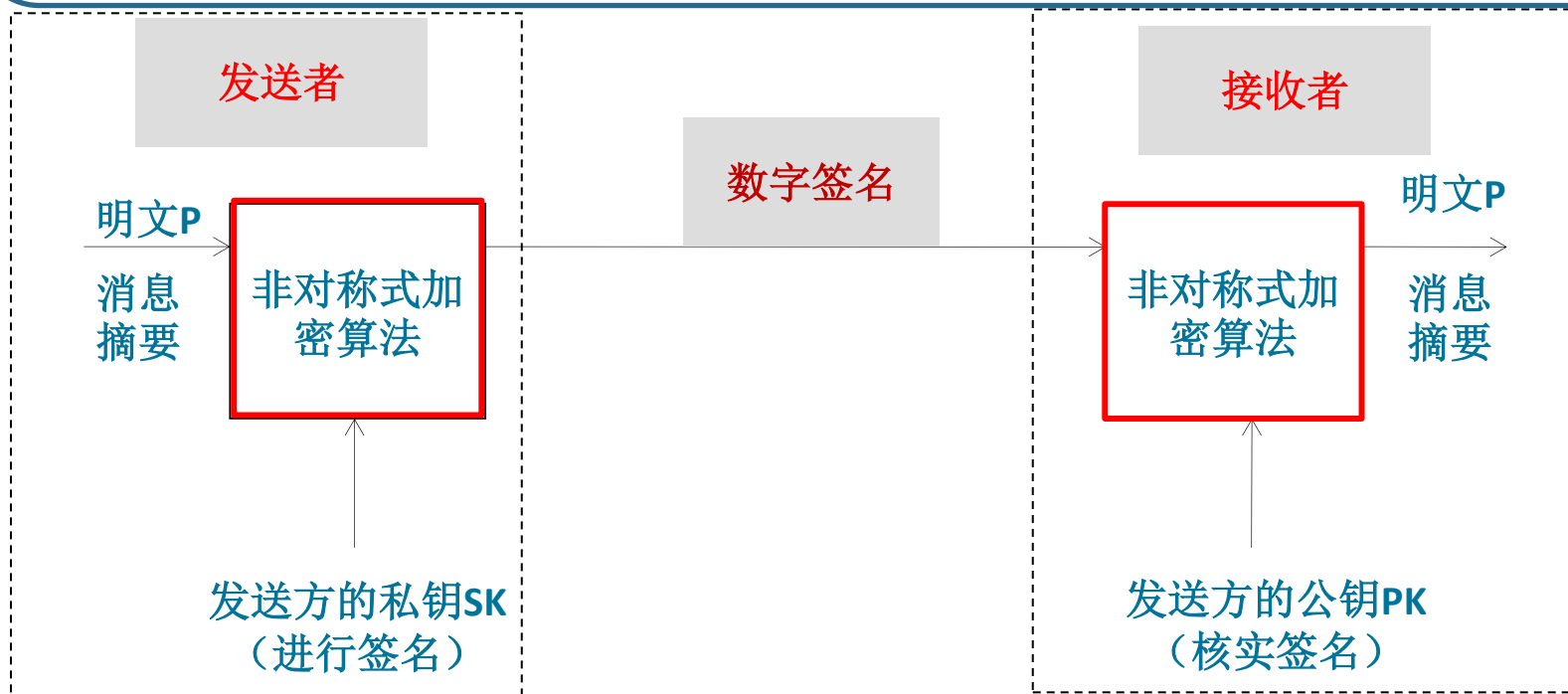
- ***椭圆加密算法 (ECC)**  是一种公钥加密体制，最初由Koblitz和Miller两人于1985年提出，其数学基础是利用椭圆曲线上的有理点构成Abel加法群上椭圆离散对数的计算困难性。
- 公钥密码体制根据其所依据的难题一般分为三类：大素数分解问题类、离散对数问题类、椭圆曲线类。有时也把椭圆曲线类归为离散对数类。
- ECC的主要优势是在某些情况下它比其他的方法**使用更小的密钥**——比如**RSA加密算法**——提供**相当的或更高等级的安全**。ECC的另一个优势是可以定义群之间的**双线性映射**；双线性映射已经在密码学中发现了大量的应用，例如基于身份的加密。不过一个缺点是加密和解密操作的实现比其他机制**花费的时间长**。

5.3 实用加密技术概述

■ 5.3.3 数字签名（侧重加密应用, 6章具体介绍）

数字签名可带来3个方面的安全性：

- 1) 消息的完整性。
- 2) 消息源确认。
- 3) 不可抵赖性。



非对称加密算法实现数字签名过程



5.3 实用加密技术概述

讨论思考

1. 比较对称加密技术和非对称加密技术。
2. 数字签名技术的原理及其应用场景

5.5 本章小结

本章小结

- 本章介绍了密码技术**相关概念**, 密码学与**密码体制**、数据及网络**加密方式**
- 讨论了密码破译**方法**与密钥**管理**;
- 概述了**实用加密技术**, 包括:
 - 对称密码体制
 - 古典对称密码
 - 现代分组密码
 - 现代流密码算法
 - 现代散列算法
 - 非对称密码体制
 - 数字签名 (综合应用散列算法和非对称密码体制)



课后作业

练习与实践5

1 (5)

3 (1) (3)

4 (1)

作业提交：手写拍照或电子文档，保存为pdf格式，命名“学号-姓名”
班长/学委打包发我QQ (97251651) 或者邮箱 (jingkun@ahu.edu.cn)



5.4 AES算法加密解密实验

在本实验中，我们将学习如何对电脑中的一个文本文件使用AES算法进行加密解密。Java语言JDK提供了丰富的密码学类库，本实验将使用Java语言来实现对文件的AES加解密，使用的AES分组密码算法的密钥长度为128比特，运行的模式为CBC模式，CBC模式的初始IV值为128位全0比特。

■ 5.4.1 实验目的与要求

通过文件加解密的实验，进一步加深对AES加密算法应用的理解，同时学习如何使用Java语言对文件进行加解密操作。

5.4.2 实验环境及学时

实验环境与设备：Windows 电脑一台，需装有Java虚拟机JDK 1.6及以上版本。

实验用时：2学时（90分钟）

5.4.3 实验步骤

- 1. 项目运行所需的头文件如下，需要导入 `javax.crypto.Cipher`, `javax.crypto.KeyGenerator`, `javax.crypto.SecretKey` 等几个密码学类库。

```
1 package javaapplication1;
2 import java.nio.charset.StandardCharsets;
3 import java.nio.file.*;
4 import java.util.Arrays;
5 import javax.crypto.Cipher;
6 import javax.crypto.KeyGenerator;
7 import javax.crypto.SecretKey;
8 import javax.crypto.spec.IvParameterSpec;
9 import javax.xml.bind.DatatypeConverter;
```

5.4.3 实验步骤

- 2. 将电脑中的文本文件 “D:\\planText1.txt” 中的所有内容读取到字节数组 bytes 中，需要保证电脑中存在该文件。

```
24 //获取电脑中文件名 “D:\\planText1.txt” 的路径
25 Path path = Paths.get("D:\\planText1.txt");
26
27 //从该文件读取内容到字节数组plainBytes1中
28 byte[] plainBytes1 = Files.readAllBytes(path);
29
30 //以16进制格式打印出字节数组plainBytes1中的内容
31 System.out.println("plainBytes1: " +
32     DatatypeConverter.printHexBinary(plainBytes1));
```


5.4.3 实验步骤

3. 设定加密算法，采用AES算法，运行模式为CBC模式，消息填充方法为PKCS5Padding，设定AES加密算法所使用的密钥参数，设定为128比特长度。生成密钥sKey，以16进制格式打印出密钥sKey中的内容。

```
33
34      /*设定加密算法，采用AES算法，运行模式为CBC模式，
35      消息填充方法为PKCS5Padding. */
36      Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
37      // 设定AES加密算法所使用的密钥参数, 设定为128比特长度
38      KeyGenerator keyGen = KeyGenerator.getInstance("AES");
39      keyGen.init(128);
40      // 生成密钥sKey
41      SecretKey sKey = keyGen.generateKey();
42      //以16进制格式打印出密钥sKey中的内容
43      System.out.println("Key: " +
44      DatatypeConverter.printHexBinary(sKey.getEncoded()));
```

5.4.3 实验步骤

4. AES算法对象cipher初始化为加密模式，所使用的密钥为sKey，CBC模式的IV值为128位全0比特，将plainBytes 加密并保存到密文字节数组 cipherBytes1中，以16进制格式打印出字节数组cipherBytes中的内容，将密文字节数组 cipherBytes 中的数据写入文件 “D:\\cipher.t”中。

```
45
46      /* AES算法对象cipher初始化为加密模式，
47      所使用的密钥为sKey，CBC模式的IV值为128位全0比特 */
48      cipher.init(Cipher. ENCRYPT_MODE, sKey,
49      |           new IvParameterSpec(new byte[16]));
50      // 将plainBytes 加密并保存到密文字节数组 cipherBytes1中
51      byte[] cipherBytes1 = cipher.doFinal(plainBytes1);
52      //以16进制格式打印出字节数组cipherBytes中的内容
53      System.out.println("cipherBytes: "+
54      |           DatatypeConverter.printHexBinary(cipherBytes1));
55      // 将 密文字节数组 cipherBytes 中的数据写入文件 "D:\\cipher.t"中。
56      Files.write(Paths.get("D:\\cipher.t"),
57      |           cipherBytes1, StandardOpenOption.CREATE);
```

5.4.3 实验步骤

- 5. 接下来实现解密过程。将密文文件内容读取到字节数组 cipherBytes2 中，AES 算法对象 cipher 初始化为解密模式，所使用的解密密钥为 sKey，CBC 模式的 IV 值为 128 位全 0 比特，将 cipherBytes2 解密并保存到明文字节数组 plainBytes2 中，以 16 进制格式打印出字节数组 plainBytes2 中的内容。

```
58
59      /*      接下来实现解密过程      */
60      //将密文文件内容读取到字节数组 cipherBytes2中
61      byte[] cipherBytes2 = Files.readAllBytes(Paths.get("D:\\cipher.t"));
62      /* AES算法对象cipher初始化为解密模式，
63         所使用的解密密钥为sKey，CBC模式的IV值为128位全0比特 */
64      cipher.init(Cipher.DECRYPT_MODE, sKey,
65                  new IvParameterSpec(new byte[16]));
66      // 将cipherBytes2解密并保存到明文字节数组 plainBytes2中
67      byte[] plainBytes2 = cipher.doFinal(cipherBytes2);
68      //以16进制格式打印出字节数组plainBytes2中的内容
69      System.out.println("plainBytes2: "+
70                          DatatypeConverter.printHexBinary(plainBytes2));
```

5.4.3 实验步骤

- 6. 判断明文字节数组plainBytes2 里的内容是否与 plainBytes 相同，将解密得到的明文使用UTF_8编码写入到“D:\\planText2.txt”，可以用记事本查看“D:\\planText1.txt”与“D:\\planText2.txt”两个文件的内容相同。

```
71
72 //判断明文字节数组plainBytes2 里的内容是否与 plainBytes相同
73 System.out.println(Arrays.equals(plainBytes1, plainBytes2));
74
75 /*将解密得到的明文使用UTF_8编码写入到 “D:\\planText2.txt”，
76 可以用记事本查看“D:\\planText1.txt” 与
77 “D:\\planText2.txt”两个文件的内容相同。*/
78 Files.write( Paths.get("D:\\planText2.txt"),
79 new String(plainBytes2, StandardCharsets.UTF_8).getBytes());
```

“十三五”国家重点出版规划
国家自然科学基金项目资助

上海市普通高校优秀教材奖
上海高校精品/优质在线课程

诚挚谢意！

网络安全技术及应用

国家智慧教育平台

国家智慧教育平台-上海精品/优质在线课程



网络安全技术

(第5版, 慕课微课版立体化)

